Team notebook

Universidad Mayor de San Simón - Club de frontón 2880: Oliver Pozo, Rodrigo Salguero, Mateo Valdivia

April 9, 2024

Contents

1 DataStructures **2**
- 1.1 kdtree . . . 2
- 1.2 lazy segment tree . . . 3
- 1.3 link-cut . . . 3
- 1.4 merge sort tree . . . 4
- 1.5 persistent range updates . . . 4
- 1.6 segment-tree-iterativo . . . 4
- 1.7 Sparse Table . . . 4
- 1.8 treap . . . 4

2 DP **5**
- 2.1 ConvexHull . . . 5
- 2.2 divide-and-conquer . . . 5
- 2.3 knuth . . . 5
- 2.4 lis . . . 5
- 2.5 matrix-fast-pow . . . 5
- 2.6 SOS . . . 6

3 fft **6**
- 3.1 fft-operations . . . 6
- 3.2 fft . . . 6
- 3.3 fht . . . 7
- 3.4 karatsuba . . . 7

4 Flows **8**
- 4.1 Algoritmo hungaro de asignacion . . . 8
- 4.2 max flow . . . 8
- 4.3 min cost flow . . . 8
- 4.4 simplex . . . 9

5 Geometry **10**
- 5.1 centroid . . . 10
- 5.2 chull . . . 10
- 5.3 circle2ptsrad . . . 10
- 5.4 halfplane . . . 10
- 5.5 latlong . . . 11
- 5.6 line . . . 11
- 5.7 minkowski . . . 11
- 5.8 nearest neighbour . . . 11
- 5.9 planar graph dual . . . 12
- 5.10 point-in-poly . . . 12

6 Graph **12**
- 6.1 2sat . . . 12
- 6.2 bellman-ford . . . 13
- 6.3 centroid . . . 13
- 6.4 dominator$_t ree$ . . . 13
- 6.5 dynamic-connectivity . . . 13
- 6.6 edmonds-blossom . . . 14

6.7 erdos gallai . . . 14
6.8 eulerian-path . . . 15
6.9 HLD LCA con heavy paths . . . 15
6.10 max weight lca . . . 15
6.11 tarjan y block cut tree . . . 15

7 Math **16**
7.1 basis z2 . . . 16
7.2 berlekamp . . . 16
7.3 catalan . . . 16
7.4 chinese-remainder . . . 16
7.5 count primes 23 . . . 17
7.6 exteded-euclid . . . 17
7.7 fast-gcd . . . 17
7.8 formulas . . . 17
7.9 gauss mod prime . . . 18
7.10 gauss . . . 18
7.11 interpol-o(n) . . . 18
7.12 josephus . . . 18
7.13 linear sieve . . . 19
7.14 LinearRecurrence . . . 19
7.15 matrix-determinant . . . 19
7.16 matrix-fast-pow . . . 19
7.17 moebius . . . 19
7.18 pollard-rho-miller-rabil . . . 20
7.19 stirling y bell . . . 20
7.20 tonelly shanks . . . 20

8 other **20**
8.1 dates . . . 20
8.2 discrete log . . . 21
8.3 harmonic lemma . . . 21
8.4 mo . . . 21
8.5 parallel binary search . . . 21
8.6 pragmas cubanos . . . 21
8.7 prime numbers . . . 22
8.8 python fast in . . . 22
8.9 python . . . 22
8.10 shortcuts . . . 23
8.11 simulate analing . . . 23
8.12 theo . . . 23

9 Strings **24**
9.1 aho-corasick . . . 24
9.2 algoritmo-z . . . 24
9.3 Dinamic Aho Corasick . . . 24
9.4 hsh-128 . . . 24
9.5 manacher . . . 25
9.6 preffix-function . . . 25
9.7 suffix-array . . . 25

1 DataStructures

1.1 kdtree

```
const int maxn=200005; struct kdtree {
  int xl,xr,yl,yr,zl,zr,max,flag; //flag=0:x axis 1:y 2:z
} tr[5000005];
int N,M,lastans,xq,yq,a[maxn],pre[maxn],nxt[maxn];
int x[maxn],y[maxn],z[maxn],wei[maxn];
int xc[maxn],yc[maxn],zc[maxn],wc[maxn],hash[maxn],biao[maxn];
bool cmp1(int a,int b) { return x[a]<x[b]; }
bool cmp2(int a,int b) { return y[a]<y[b]; }
bool cmp3(int a,int b) { return z[a]<z[b]; }
void makekdtr(int node,int l,int r,int flag) {
  if (l>r) { tr[node].max=-INFL; return; }
  int xl=INFL,xr=-INFL,yl=INFL,yr=-INFL,zl=INFL,zr=-INFL,maxc=-INFL;
  for (int i=l;i<=r;i++)
    xl=min(xl,x[i]),xr=max(xr,x[i]),yl=min(yl,y[i]),yr=max(yr,y[i]),
    zl=min(zl,z[i]),zr=max(zr,z[i]),maxc=max(maxc,wei[i]),
    xc[i]=x[i],yc[i]=y[i],zc[i]=z[i],wc[i]=wei[i],biao[i]=i;
  tr[node].flag=flag; tr[node].xl=xl,tr[node].xr=xr,tr[node].yl=yl;
  tr[node].yr=yr,tr[node].zl=zl,tr[node].zr=zr; tr[node].max=maxc;
  if (l==r) return; if (flag==0) sort(biao+l,biao+r+1,cmp1);
  if (flag==1) sort(biao+l,biao+r+1,cmp2);
  if (flag==2) sort(biao+l,biao+r+1,cmp3);for (int i=l;i<=r;i++)
x[i]=xc[biao[i]],y[i]=yc[biao[i]],z[i]=zc[biao[i]],wei[i]=wc[biao[i]];
  makekdtr(node*2,l,(l+r)/2,(flag+1)%3);
  makekdtr(node*2+1,(l+r)/2+1,r,(flag+1)%3);}
int getmax(int node,int xl,int xr,int yl,int yr,int zl,int zr) {
  xl=max(xl,tr[node].xl);xr=min(xr,tr[node].xr);
  yl=max(yl,tr[node].yl);yr=min(yr,tr[node].yr);
  zl=max(zl,tr[node].zl);zr=min(zr,tr[node].zr);
  if (tr[node].max==-INFL) return 0;
  if ((xr<tr[node].xl)||(xl>tr[node].xr)) return 0;
```

```
  if ((yr<tr[node].yl)||(yl>tr[node].yr)) return 0;
  if ((zr<tr[node].zl)||(zl>tr[node].zr)) return 0;
  if ((tr[node].xl==xl)&&(tr[node].xr==xr)&&
    (tr[node].yl==yl)&&(tr[node].yr==yr)&&
    (tr[node].zl==zl)&&(tr[node].zr==zr)) return tr[node].max;
  else return max(getmax(node*2,xl,xr,yl,yr,zl,zr),
       getmax(node*2+1,xl,xr,yl,yr,zl,zr)); }//N 3Drect with weights
// find the maximum weight containing the given 3D-point
```

1.2 lazy segment tree

```
int ar[tam], t[4 * tam], l[4 * tam];
void push(int b, int e, int node) {
  if(l[node]) { t[node] += l[node]; if(b < e)
      l[node * 2 + 1] += l[node],l[node*2+2]+=l[node];l[node]=0;}}
void update(int b, int e, int node, int i, int j, int val) {
  if(b > e) return; push(b, e, node);
  if(e < i || b > j) return; if(b >= i && e <= j) {
    l[node] += val; push(b, e, node); return; }
  index; update(b, mid, l, i, j, val);
  update(mid + 1, e, r, i, j, val); t[node] = max(t[l], t[r]); }
```

1.3 link-cut

```
const int N_DEL = 0, N_VAL = 0; //delta, value
inline int mOp(int x, int y){return x+y;}//modify
inline int qOp(int lval, int rval){return lval + rval;}//query
inline int dOnSeg(int d, int len){return d==N_DEL ? N_DEL : d*len;}
//mostly generic
inline int joinD(int d1, int d2){
  if(d1==N_DEL)return d2;if(d2==N_DEL)return d1;return mOp(d1, d2);}
inline int joinVD(int v, int d){return d==N_DEL ? v : mOp(v, d);}
struct Node_t{
  int sz, nVal, tVal, d; bool rev; Node_t *c[2], *p;
  Node_t(int v) : sz(1), nVal(v), tVal(v), d(N_DEL), rev(0), p(0){
  c[0]=c[1]=0; }
  bool isRoot(){return !p || (p->c[0] != this && p->c[1] != this);}
  void push(){
  if(rev){rev=0; swap(c[0], c[1]);
    fore(x,0,2)if(c[x])c[x]->rev^=1; }
  nVal=joinVD(nVal, d); tVal=joinVD(tVal, dOnSeg(d, sz));
  fore(x,0,2)if(c[x])c[x]->d=joinD(c[x]->d, d);d=N_DEL; }void upd();};
typedef Node_t* Node;
int getSize(Node r){return r ? r->sz : 0;} int getPV(Node r){
  return r ? joinVD(r->tVal, dOnSeg(r->d,r->sz)) : N_VAL;}
void Node_t::upd(){
  tVal = qOp(qOp(getPV(c[0]), joinVD(nVal, d)), getPV(c[1]));
  sz = 1 + getSize(c[0]) + getSize(c[1]); }
void conn(Node c, Node p, int il){if(c)c->p=p;if(il>=0)p->c[!il]=c;}
void rotate(Node x){ Node p = x->p, g = p->p;
  bool gCh=p->isRoot(), isl = x==p->c[0];
  conn(x->c[isl],p,isl); conn(p,x,!isl);
  conn(x,g,gCh?-1:(p==g->c[0])); p->upd(); }
void spa(Node x){//splay
  while(!x->isRoot()){Node p = x->p, g = p->p;
  if(!p->isRoot())g->push();p->push(); x->push();
  if(!p->isRoot())rotate((x==p->c[0])==(p==g->c[0])? p : x);rotate(x);}
  x->push(); x->upd();}
Node exv(Node x){Node last=0;
  for(Node y=x; y; y=y->p)spa(y),y->c[0]=last,y->upd(),last=y;
  spa(x); return last;}
void mkR(Node x){exv(x);x->rev^=1;}//makeRoot
Node getR(Node x){exv(x);while(x->c[1])x=x->c[1];spa(x);return x;}
Node lca(Node x, Node y){exv(x); return exv(y);}
bool connected(Node x, Node y){exv(x);exv(y); return x==y?1:x->p!=0;}
void link(Node x, Node y){mkR(x); x->p=y;}
void cut(Node x, Node y){mkR(x); exv(y); y->c[1]->p=0; y->c[1]=0;}
Node father(Node x){
  exv(x); Node r=x->c[1]; if(!r)return 0;
  while(r->c[0])r=r->c[0]; return r;}
void cut(Node x){ // cuts x from father keeping tree root
  exv(father(x));x->p=0;}
int query(Node x, Node y){mkR(x); exv(y); return getPV(y);}
void modify(Node x, Node y, int d){mkR(x);exv(y);y->d=joinD(y->d,d);}
Node lift_rec(Node x, int t){if(!x)return 0;
  if(t==getSize(x->c[0])){spa(x);return x;}
  if(t<getSize(x->c[0]))return lift_rec(x->c[0],t);
  return lift_rec(x->c[1],t-getSize(x->c[0])-1);}
Node lift(Node x, int t){ // t-th ancestor of x (lift(x,1) is x's father)
  exv(x);return lift_rec(x,t);}
int depth(Node x){ // distance from x to its tree root
  exv(x);return getSize(x)-1;}
```

1.4 merge sort tree

```
vi t[4*tam]; int ar[tam];
void build(int node, int b, int e) {
  if (b == e) { t[node].pb(ar[b]); return; }
  build(2*node, b, (b+e)/2);
  build(2*node+1, (b+e)/2+1, e);
  merge(t[2*node].begin(), t[2*node].end(),
    t[2*node+1].begin(), t[2*node+1].end(),back_inserter(t[node]));}
int query(int node, int b, int e, int i, int j, int x, int y) {
  if (j < b || i > e) return 0; if (i <= b && j >= e)
  return upper_bound(t[node].begin(), t[node].end(), y) -
      lower_bound(t[node].begin(), t[node].end(), x);
  return query(2*node, b, (b+e)/2, i, j, x, y) +
     query(2*node+1, (b+e)/2+1, e, i, j, x, y); }
```

1.5 persistent range updates

```
bool hasflag[]; int flag[];
int newlazykid(int node, int delta, int L, int R);
void propagate(int p, int L, int R) {if (hasflag[p]) {
    if (L != R) {
      l[p] = newlazykid(l[p], flag[p], L, M);
      r[p] = newlazykid(r[p], flag[p], M + 1, R);}
    hasflag[p] = false;}}
int newlazykid(int node, int delta, int L, int R) {
  int p = ++NODES;l[p] = l[node];r[p]=r[node];flag[p]=flag[node];
  hasflag[p] = true;flag[p] += delta;
  st[p] = st[node] + (R - L + 1) * delta;return p;}
int update(int a, int b, int x, int p, int L=0, int R=n-1) {
  if (b < L || R < a) return p;
  if (a <= L && R <= b) return newlazykid(p, x, L, R);
  propagate(p, L, R);
  return newparent(update(a, b, l[p], x, L, M),
          update(a, b, r[p], x, M + 1, R));}
int query(int a, int b, int p, int L=0, int R=n-1) {
  if (b < L || R < a) return 0;
  if (a <= L && R <= b) return st[p];
  propagate(p, L, R);
  return query(a, b, l[p], x, L, M) + query(a b,r[p],x,M+1,R);}
```

1.6 segment-tree-iterativo

```
int t[2 * tam]; void build() {
  for (int i = n - 1; i > 0; --i) t[i] = t[i<<1] + t[i<<1|1]; }
void modify(int p, int value) { // set value at position p
  for (t[p += n] = value; p > 1; p >>= 1) t[p>>1] = t[p] + t[p^1]; }
int query(int l, int r) {/*sum on interval [l, r)*/int res = 0;
  for (l += n, r += n; l < r; l >>= 1, r >>= 1) {if (l&1) res+=t[l++];
  if (r&1) res += t[--r]; }return res; }
```

1.7 Sparse Table

```
int ar[tam], table[logTam][tam];
void inispar() {
  fore(i, 0, n) table[0][i] = ar[i];
  for(int k = 0; (1 << k) < n; k++)
    for(int i = 0; i + (1 << k) < n; i++)
      table[k + 1][i] = min(table[k][i], table[k][i + (1 << k)]); }
int query(int b, int e) {
  int lev = 31 - __builtin_clz(e - b + 1);
  return min(table[lev][b], table[lev][e - (1 << lev) + 1]); }
```

1.8 treap

```
struct item {
  int key, pri, siz;
  item *l, *r; item() {}
  item(int key) : key(key), siz(1), pri(rand()), l(0), r(0) {} };
typedef item* pitem;
int sz(pitem t) {
  return (t?t->siz:0); }
void up_sz(pitem t) {
  if(t) t->siz = sz(t->l) + 1 + sz(t->r); }
void split(pitem t, pitem &l, pitem &r, int val) {
  if(!t) r = l = NULL;
  else if(t->key < val) split(t->r, t->r, r, val), l = t;
  else split(t->l, l, t->l, val), r = t; up_sz(t); }
void merge(pitem &t, pitem l, pitem r) {
  if(!l || !r) t=(l?l:r);
  else if(l->pri >= r->pri) merge(l->r, l->r, r), t = l;
  else merge(r->l, l, r->l),t=r; up_sz(t); }
```

2 DP

2.1 ConvexHull

```
// kx+m, and query maximum values at points x.
#pragma once
struct Line { mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; } };
struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) { x->p = inf; return false; }
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);return x->p>=y->p;}
  void add(ll k, ll m) {auto z = insert({k, m, 0}), y = z++, x=y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p>=y->p)isect(x,erase(y));}
  ll query(ll x) { assert(!empty()); auto l = *lower_bound(x);
    return l.k * x + l.m; } };
```

2.2 divide-and-conquer

```
//DP[i][j]=min(DP[i-1][k]+C[k][j]); K[i][j]<=K[i][j+1]
ll lastDP[tam], DP[tam]; int C[tam][tam];
void DC(int b, int e, int KL, int KR) {
  int mid = (b + e) / 2; pair<ll, int> best = mp(-1, KL);
  for (int k = KL; k < min(mid, KR+1); k++)
  best = max( best, mp(lastDP[k] + C[k+1][mid], k) );
  DP[mid] = best.first; int K = best.second;
  if (b <= mid-1) DC(b, mid-1, KL, K);
  if (mid+1 <= e) DC(mid+1, e, K, KR); }
```

2.3 knuth

```
/*Basado en: Minimo costo de cortar una barra en K lugares pre-definidos
Costo de corte: largo de la barra que se esta cortando
A[i] -> posicion del i-esimo corte, A[0] = 0, A[n-1] = Largo total
DP[i][j] = min( DP[i][k] + DP[k][j] ) + C[i][j],
para i <= k <= j donde C[i][j] es el largo de la barra A[i] <-> A[j]
K[i][j-1] <= K[i][j] <= K[i+1][j]*/
fore(sz, 0, n) {
  for (int i = 0; i + sz < n; i++) {
  int j = i+sz; // CASOS BASE
  if (sz <= 1) { // Barra inexistente o con cero cortes en medio
    DP[i][j] = 0; continue; }
  if (sz == 2) { // Barra con un solo corte posible en medio
    K[i][j] = i+1; DP[i][j] = C[i][j]; continue; }
  int KL = K[i][j-1]; int KR = K[i+1][j]; DP[i][j] = INF;
  for (int k = KL; k <= KR; k++) {
    int newVal = DP[i][k] + DP[k][j] + C[i][j];
    if (newVal < DP[i][j]) {K[i][j] = k; DP[i][j] = newVal;}}}}
```

2.4 lis

```
for (int i = 0; i < n; ++i) {
  // increasing: lower_bound; non-decreasing: upper_bound
  int j = lower_bound(dp, dp + lis, v[i]) - dp;
  dp[j] = min(dp[j], v[i]); lis = max(lis, j + 1); }
```

2.5 matrix-fast-pow

```
// kth term of linear recurrence
// of size m a_i = sum(a_(i-j)*p_j)
// f(x) = x^m - sum(x^(m-j)*p_j)
// g(x^k) = g(x^k mod f)
typedef vector<vector<ll> > Matrix;
Matrix ones(int n) {
  Matrix r(n,vector<ll>(n)); fore(i,0,n)r[i][i]=1; return r; }
Matrix operator*(Matrix &a, Matrix &b) {
  int n=SZ(a),m=SZ(b[0]),z=SZ(a[0]); Matrix r(n,vector<ll>(m));
  fore(i,0,n)fore(j,0,m)fore(k,0,z)
  r[i][j]+=a[i][k]*b[k][j],r[i][j]%=mod; return r; }
Matrix be(Matrix b, ll e) {Matrix r=ones(SZ(b));
```

```
  while(e){if(e&1LL)r=r*b;b=b*b;e/=2;} return r; }
```

2.6 SOS

```
//iterative version
for(int mask = 0; mask < (1<<N); ++mask){
  dp[mask][-1] = A[mask]; //handle base case separately (leaf states)
  for(int i = 0;i < N; ++i){
  if(mask & (1<<i)) dp[mask][i] = dp[mask][i-1] + dp[mask^(1<<i)][i-1];
  else dp[mask][i] = dp[mask][i-1]; }
  F[mask] = dp[mask][N-1]; }
//memory optimized, super easy to code.
for(int i = 0; i<(1<<N); ++i) F[i] = A[i];
for(int i = 0;i < N; ++i) for(int mask = 0; mask < (1<<N); ++mask){
  if(mask & (1<<i)) F[mask] += F[mask^(1<<i)]; }
```

3 fft

3.1 fft-operations

```
//Polynomial division: O(n*log(n))
//Multi-point polynomial evaluation: O(n*log^2(n))
//Polynomial interpolation: O(n*log^2(n))
//Works with NTT. For FFT, replace addmod,submod,mulmod,inv
poly add(poly &a, poly &b){
  int n=sz(a),m=sz(b);poly ans(max(n,m));fore(i,0,max(n,m)){
    if(i<n) ans[i]=addmod(ans[i],a[i]);
    if(i<m) ans[i]=addmod(ans[i],b[i]); }
  while(sz(ans)>1&&!ans.back())ans.pop_back(); return ans; }
poly invert(poly &b, int d){
  poly c = {inv(b[0])};
  while(sz(c)<=d){int j=2*sz(c); auto bb=b; bb.resize(j);
    poly cb=multiply(c,bb);
    fore(i,0,sz(cb)) cb[i]=submod(0,cb[i]); cb[0]=addmod(cb[0],2);
    c=multiply(c,cb); c.resize(j);}c.resize(d+1); return c; }
pair<poly,poly> divslow(poly &a, poly &b){poly q,r=a;
  while(sz(r)>=sz(b)){ q.pb(mulmod(r.back(),inv(b.back())));
    if(q.back()) fore(i,0,sz(b)){
      r[sz(r)-i-1]=submod(r[sz(r)-i-1],mulmod(q.back(),b[sz(b)-i-1])); }
    r.pop_back(); }
  reverse(all(q)); return {q,r}; }
pair<poly,poly> divide(poly &a, poly &b){ //returns {quotient,remainder}
  int m=sz(a),n=sz(b),MAGIC=750; if(m<n) return {{0},a};
  if(min(m-n,n)<MAGIC)return divslow(a,b);
  poly ap=a; reverse(all(ap));
  poly bp=b; reverse(all(bp)); bp=invert(bp,m-n);
  poly q=multiply(ap,bp); q.resize(sz(q)+m-n-sz(q)+1,0);
  reverse(all(q)); poly bq=multiply(b,q);
  fore(i,0,sz(bq)) bq[i]=submod(0,bq[i]); poly r=add(a,bq);
  return {q,r}; } vector<poly> tree;
void filltree(vector<tf> &x){ int k=sz(x); tree.resize(2*k);
  fore(i,k,2*k) tree[i]={submod(0,x[i-k]),1};
  for(int i=k-1;i;i--) tree[i]=multiply(tree[2*i],tree[2*i+1]); }
vector<tf> evaluate(poly &a, vector<tf> &x){
  filltree(x); int k=sz(x); vector<poly> ans(2*k);
  ans[1]=divide(a,tree[1]).snd;
  fore(i,2,2*k) ans[i]=divide(ans[i>>1],tree[i]).snd;
  vector<tf> r; fore(i,0,k) r.pb(ans[i+k][0]); return r; }
poly derivate(poly &p){poly ans(sz(p)-1);
  fore(i,1,sz(p)) ans[i-1]=mulmod(p[i],i); return ans; }
poly interpolate(vector<tf> &x, vector<tf> &y){
  filltree(x); poly p=derivate(tree[1]); int k=sz(y);
  vector<tf> d=evaluate(p,x); vector<poly> intree(2*k);
  fore(i,k,2*k) intree[i]={mulmod(y[i-k],inv(d[i-k]))};
  for(int i=k-1;i;i--){
    poly p1=multiply(tree[2*i],intree[2*i+1]);
    poly p2=multiply(tree[2*i+1],intree[2*i]);
    intree[i]=add(p1,p2); } return intree[1]; }
```

3.2 fft

```
// MAXN must be power of 2 !!
// MOD-1 needs to be a multiple of MAXN !!
// big mod and primitive root for NTT:
const int MAXN = 1<<20;typedef ll tf;
tf addmod(tf a, tf b){tf r=a+b;if(r>=MOD)r-=MOD;return r;}
tf submod(tf a, tf b){tf r=a-b;if(r<0)r+=MOD;return r;}
tf mulmod(ll a, ll b){return a*b%MOD;}
tf inv(tf a){return pot(a,MOD-2);}
typedef vector<tf> poly; const tf MOD=998244353, RT=3;
// const tf MOD=2305843009255636993,RT=5;
// FFT
// struct CD { double r,i;
```

```
//   CD(double r=0, double i=0):r(r),i(i){}
//   double real()const{return r;}
//   void operator/=(const int c){r/=c, i/=c;} };
// CD operator*(const CD& a, const CD& b){
//   return CD(a.r*b.r-a.i*b.i,a.r*b.i+a.i*b.r);}
// CD operator+(const CD& a, const CD& b){return CD(a.r+b.r,a.i+b.i);}
// CD operator-(const CD& a, const CD& b){return CD(a.r-b.r,a.i-b.i);}
// const double pi=acos(-1.0);
// NTT
struct CD { tf x; CD(tf x):x(x){} CD(){} };
CD operator*(const CD& a, const CD& b){return CD(mulmod(a.x,b.x));}
CD operator+(const CD& a, const CD& b){return CD(addmod(a.x,b.x));}
CD operator-(const CD& a, const CD& b){return CD(submod(a.x,b.x));}
vector<tf> rts(MAXN+9,-1);
CD root(int n, bool inv){
  tf r=rts[n]<0?rts[n]=pot(RT,(MOD-1)/n):rts[n];
  return CD(inv?pot(r,MOD-2):r); }
CD cp1[MAXN+9],cp2[MAXN+9]; int R[MAXN+9];
void dft(CD* a, int n, bool inv){
  fore(i,0,n)if(R[i]<i)swap(a[R[i]],a[i]);
  for(int m=2;m<=n;m*=2){
    // double z=2*pi/m*(inv?-1:1); // FFT
    // CD wi=CD(cos(z),sin(z)); // FFT
    CD wi=root(m,inv); // NTT
    for(int j=0;j<n;j+=m){ CD w(1);
      for(int k=j,k2=j+m/2;k2<j+m;k++,k2++){
        CD u=a[k];CD v=a[k2]*w;a[k]=u+v;a[k2]=u-v;w=w*wi; } } }
  // if(inv)fore(i,0,n)a[i]/=n; // FFT
  if(inv){ // NTT
    CD z(pot(n,MOD-2)); fore(i,0,n)a[i]=a[i]*z; } }
poly multiply(poly& p1, poly& p2){
  int n=p1.size()+p2.size()+1, m=1,cnt=0;
  while(m<=n)m+=m,cnt++;
  fore(i,0,m){R[i]=0;fore(j,0,cnt)R[i]=(R[i]<<1)|((i>>j)&1);}
  fore(i,0,m)cp1[i]=0,cp2[i]=0;
  fore(i,0,p1.size())cp1[i]=p1[i];
  fore(i,0,p2.size())cp2[i]=p2[i];
  dft(cp1,m,false);dft(cp2,m,false);
  fore(i,0,m)cp1[i]=cp1[i]*cp2[i]; dft(cp1,m,true);
  poly res; n-=2;
  // fore(i,0,n)res.pb((tf)floor(cp1[i].real()+0.5)); // FFT
  fore(i,0,n)res.pb(cp1[i].x); // NTT
  return res; }
```

3.3 fht

```
ll c1[tam+9],c2[tam+9]; // tam must be power of 2 !!
void fht(ll* p, int n, bool inv){
  for(int l = 1; 2 * l <= n; l *= 2)
  for(int i = 0; i < n; i += 2 * l)
  fore(j, 0, l) { ll u = p[i + j], v = p[i + l + j];
    if(!inv) p[i + j] = u + v, p[i + l + j] = u - v; // XOR
    else p[i + j] = (u + v) / 2,p[i + l + j] = (u - v) / 2;
    //if(!inv) p[i + j] = v, p[i + l + j] = u + v; // AND
    //else p[i + j] = -u + v, p[i + l + j] = u;
    //if(!inv) p[i + j] = u + v, p[i + l + j] = u; // OR
    //else p[i + j] = v, p[i + l + j] = u - v;
  } }
// like polynomial multiplication, but XORing exponents
// instead of adding them (also ANDing, ORing)
vector<ll> multiply(vector<ll>& p1, vector<ll>& p2){
  int n = 1<<(32-__builtin_clz(max(sz(p1), sz(p2)) - 1));
  fore(i, 0, n) c1[i] = 0, c2[i] = 0; fore(i,0,sz(p1))c1[i]=p1[i];
  fore(i, 0, sz(p2)) c2[i] = p2[i];fht(c1, n, false);fht(c2,n,false);
  fore(i, 0, n) c1[i] *= c2[i]; fht(c1, n,true);return vi(c1,c1+n);}
void fht(vector<ll>& p, bool inv) {
fore(i,0,sz(p))c1[i]=p[i];fht(c1,sz(p),inv);fore(i,0,sz(p))p[i]=c1[i];}
```

3.4 karatsuba

```
// #define add(n,s,d,k) fore(i,0,n)(d)[i]+=(s)[i]*k
#define add(n,s,d,k) fore(i,0,n)(d)[i]+=(s)[i]*k%MOD, (d)[i] = ((d)[i] %
    MOD + MOD) % MOD;
ll* ini(int n){ll *r=new ll[n];fill(r,r+n,0);return r;}
void karatsura(int n, ll* p, ll* q, ll* r){
  if(n<=0)return;
  // if(n<35)fore(i,0,n)fore(j,0,n)r[i+j]+=p[i]*q[j];
  if(n<35)fore(i,0,n)fore(j,0,n)r[i+j]+=p[i]*q[j] % MOD, r[i + j] %= MOD;
  else {
    int nac=n/2,nbd=n-n/2;
    ll *a=p,*b=p+nac,*c=q,*d=q+nac;
    ll *ab=ini(nbd+1),*cd=ini(nbd+1),*ac=ini(nac*2),*bd=ini(nbd*2);
    add(nac,a,ab,1);add(nbd,b,ab,1);
    add(nac,c,cd,1);add(nbd,d,cd,1);
    karatsura(nac,a,c,ac);karatsura(nbd,b,d,bd);
    add(nac*2,ac,r+nac,-1);add(nbd*2,bd,r+nac,-1);
    add(nac*2,ac,r,1);add(nbd*2,bd,r+nac*2,1);
```

```
    karatsura(nbd+1,ab,cd,r+nac);
    free(ab);free(cd);free(ac);free(bd); } }
vector<ll> multiply(vector<ll> p0, vector<ll> p1){
  int n=max(p0.size(),p1.size()); ll *p=ini(n),*q=ini(n),*r=ini(2*n);
  fore(i,0,p0.size())p[i]=p0[i]; fore(i,0,p1.size())q[i]=p1[i];
  karatsura(n,p,q,r); vector<ll> rr(r,r+p0.size()+p1.size()-1);
  free(p);free(q);free(r); return rr; }
```

4 Flows

4.1 Algoritmo hungaro de asignacion

```
int n, m; int a[n+1][m+1]; // matriz de costos, 1-index
vector<int> u(n+1), v(m+1), p(m+1), way(m+1);
for (int i = 1; i <= n; ++i) {
  p[0] = i; int j0 = 0;
  vector<int> minv(m + 1, INF); vector<char> used(m + 1, false);
  do {
    used[j0] = true; int i0 = p[j0], delta = INF, j1;
    for (int j = 1; j <= m; ++j)
      if ( !used[j] ) {
        int cur = a[i0][j] - u[i0] - v[j];
        if ( cur < minv[j] ) minv[j] = cur, way[j] = j0;
        if ( minv[j] < delta ) delta = minv[j], j1 = j; }
    for (int j = 0; j <= m; ++j)
      if ( used[j] ) u[ p[j] ] += delta, v[j] -= delta;
      else minv[j] -= delta;
    j0 = j1;
  } while ( p[j0] != 0 );
  do {
    int j1 = way[j0]; p[j0] = p[j1]; j0 = j1;
  } while ( j0 ); }
vector<int> ans(n+1); // Para cada fila, en que columna esta el resultado.
for (int j = 1; j <= m; ++j) ans[ p[j] ] = j; int cost = -v[0];
```

4.2 max flow

```
struct Dinitz{
  const int INF = 1e9 + 7; Dinitz(){}
  Dinitz(int n, int s, int t) {init(n, s, t);}
  void init(int n, int s, int t) {
    S = s, T = t; nodes = n;
    G.clear(), G.resize(n); Q.resize(n); }
  struct flowEdge { int to, rev, f, cap; };
  vector<vector<flowEdge> > G;
  void addEdge(int st, int en, int cap) {
    flowEdge A = {en, (int)G[en].size(), 0, cap};
    flowEdge B = {st, (int)G[st].size(), 0, 0};
    G[st].pb(A); G[en].pb(B); }
  int nodes, S, T; vi work, lvl, Q;
  bool bfs() {
    int qt = 0; Q[qt++] = S;
    lvl.assign(nodes, -1); lvl[S] = 0;
    for (int qh = 0; qh < qt; qh++) {
      int v = Q[qh];
      for (flowEdge &e : G[v]) {
        int u = e.to;
        if (e.cap <= e.f || lvl[u] != -1) continue;
        lvl[u] = lvl[v] + 1; Q[qt++] = u; } }
    return lvl[T] != -1; }
  int dfs(int v, int f) {
    if (v == T || f == 0) return f;
    for (int &i = work[v]; i < G[v].size(); i++) {
      flowEdge &e = G[v][i]; int u = e.to;
      if (e.cap <= e.f || lvl[u] != lvl[v] + 1) continue;
      int df = dfs(u, min(f, e.cap - e.f));
      if (df) {
        e.f += df; G[u][e.rev].f -= df; return df; } }
    return 0; }
  int maxFlow() {
    int flow = 0;
    while (bfs()) {
      work.assign(nodes, 0);
      while (true) {
        int df = dfs(S, INF); if (df == 0) break;
        flow += df; } }
    return flow; }
};
```

4.3 min cost flow

```
// O(min(E^2 V ^2,   EVFLOW   ))
struct CheapDinitz{
```

```
const int INF = 1e9 + 7; CheapDinitz() {}
CheapDinitz(int n, int s, int t) {init(n, s, t);}
int nodes, S, T; vi dist, pot, curFlow, prevNode, prevEdge, Q, inQue;
struct flowEdge{ int to, rev, flow, cap, cost; };
vector<vector<flowEdge>> G;
void init(int n, int s, int t) {
  nodes = n, S = s, T = t;
  curFlow.assign(n, 0), prevNode.assign(n, 0), prevEdge.assign(n, 0);
  Q.assign(n, 0), inQue.assign(n, 0); G.clear(); G.resize(n); }
void addEdge(int s, int t, int cap, int cost) {
  flowEdge a = {t, (int)G[t].size(), 0, cap, cost};
  flowEdge b = {s, (int)G[s].size(), 0, 0, -cost};
  G[s].pb(a); G[t].pb(b); }
void bellmanFord() {
  pot.assign(nodes, INF); pot[S] = 0; int qt = 0; Q[qt++] = S;
  for (int qh = 0; (qh - qt) % nodes != 0; qh++) {
    int u = Q[qh % nodes]; inQue[u] = 0;
    for (int i = 0; i < (int)G[u].size(); i++) {
      flowEdge &e = G[u][i];
      if (e.cap <= e.flow) continue;
      int v = e.to; int newDist = pot[u] + e.cost;
      if (pot[v] > newDist) {
        pot[v] = newDist;
        if (!inQue[v]) { Q[qt++ % nodes] = v; inQue[v] = 1; }
      } } } }
ii MinCostFlow() {
  bellmanFord(); int flow = 0; int flowCost = 0;
  while (true) {
    set<ii> s; s.insert({0, S}); dist.assign(nodes, INF);
    dist[S] = 0; curFlow[S] = INF;
    while (s.size() > 0) {
      int u = s.begin() -> s; int actDist = s.begin() -> f;
      s.erase(s.begin());
      if (actDist > dist[u]) continue;
      for (int i = 0; i < (int)G[u].size(); i++) {
        flowEdge &e = G[u][i]; int v = e.to;
        if (e.cap <= e.flow) continue;
        int newDist = actDist + e.cost + pot[u] - pot[v];
        if (newDist < dist[v]) {
          dist[v] = newDist; s.insert({newDist, v});
          prevNode[v] = u; prevEdge[v] = i;
          curFlow[v] = min(curFlow[u], e.cap - e.flow);
        } } }
    if (dist[T] == INF) break;
    for (int i = 0; i < nodes; i++)
      pot[i] += dist[i];
    int df = curFlow[T]; flow += df;
    for (int v = T; v != S; v = prevNode[v]) {
      flowEdge &e = G[prevNode[v]][prevEdge[v]];
      e.flow += df; G[v][e.rev].flow -= df;
      flowCost += df * e.cost; } }
  return {flow, flowCost}; }
};
```

4.4 simplex

```
vector<int> X,Y; vector<vector<double> > A;
vector<double> b,c; double z; int n, m;
void pivot(int x,int y){
  swap(X[y],Y[x]); b[x]/=A[x][y];
  fore(i,0,m)if(i!=y)A[x][i]/=A[x][y]; A[x][y]=1/A[x][y];
  fore(i,0,n)if(i!=x&&abs(A[i][y])>EPS){
  b[i]-=A[i][y]*b[x];
  fore(j,0,m)if(j!=y)A[i][j]-=A[i][y]*A[x][j];
  A[i][y]=-A[i][y]*A[x][y]; }
  z+=c[y]*b[x]; fore(i,0,m)if(i!=y)c[i]-=c[y]*A[x][i];
  c[y]=-c[y]*A[x][y]; }
pair<double,vector<double> > simplex( // maximize c^T x s.t. Ax<=b, x>=0
  vector<vector<double> > _A, vector<double> _b, vector<double> _c){
  // returns pair (maximum value, solution vector)
  A=_A;b=_b;c=_c; n=b.size();m=c.size();z=0.;
  X=vector<int>(m);Y=vector<int>(n);
  fore(i,0,m)X[i]=i; fore(i,0,n)Y[i]=i+m;
  while(1){
  int x=-1,y=-1; double mn=-EPS;
  fore(i,0,n)if(b[i]<mn)mn=b[i],x=i; if(x<0)break;
  fore(i,0,m)if(A[x][i]<-EPS){y=i;break;}
  assert(y>=0); // no solution to Ax<=b
  pivot(x,y); }
  while(1){
  double mx=EPS; int x=-1,y=-1;
  fore(i,0,m)if(c[i]>mx)mx=c[i],y=i; if(y<0)break; double mn=1e200;
  fore(i,0,n)if(A[i][y]>EPS&&b[i]/A[i][y]<mn)mn=b[i]/A[i][y],x=i;
  assert(x>=0); // c^T x is unbounded
  pivot(x,y); }
  vector<double> r(m); fore(i,0,n)if(Y[i]<m)r[Y[i]]=b[i];
  return mp(z,r); }
```

5 Geometry

5.1 centroid

```
// calcula el centro de masa de un poligono antihorario
point cen(vector<point> p) {
  double x = 0, y = 0, area = 0, ax; int n = p.size()-1;
  fore(i, 0, n) {
    ax = (p[i] ^ p[i+1]) / 2; area += ax;
    x += ax * (p[i].x + p[i+1].x) / 3;
    y += ax * (p[i].y + p[i+1].y) / 3; }
  return point(x / area, y / area); }
```

5.2 chull

```
// devuelve horario
vector<point> hull(vector<point> p) {
  int n = p.size(); vector<point> h; sort(all(p));
  fore(i, 0, n) {
  while(h.size() >= 2 && p[i].left(h[sz(h) - 2], h.back())) h.pop_back();
  h.push_back(p[i]); } h.pop_back(); int k = h.size();
  for(int i = n-1; i > -1; i--) {
  while(h.size() >= k + 2 && p[i].left(h[sz(h) - 2], h.back()))
      h.pop_back();
  h.pb(p[i]); }
  h.pop_back(); return h; }
```

5.3 circle2ptsrad

```
bool circle2PtsRad(point a, point b, double r, point &c) {//dados 2
    puntos y un radio
  double det = (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y);
  det = r * r / det - 0.25; if(det < 0.0) return false;
  det = sqrt(det); c.x = (a.x + b.x) * 0.5 + (b.y-a.y) * det;
  c.y = (a.y + b.y) * 0.5 + (a.x-b.x) * det; return true; }
```

5.4 halfplane

```
const double DINF=1e100;
struct pt {
  double x,y;
  pt(double x, double y):x(x),y(y){} pt(){}
  double norm2(){return *this**this;}
  double norm(){return sqrt(norm2());}
  bool operator==(pt p){return abs(x-p.x)<=EPS&&abs(y-p.y)<=EPS;}
  pt operator+(pt p){return pt(x+p.x,y+p.y);}
  pt operator-(pt p){return pt(x-p.x,y-p.y);}
  pt operator*(double t){return pt(x*t,y*t);}
  pt operator/(double t){return pt(x/t,y/t);}
  double operator*(pt p){return x*p.x+y*p.y;}
  double angle(pt p){ // redefine acos for values out of range
  return acos(*this*p/(norm()*p.norm()));}
  pt unit(){return *this/norm();}
  double operator%(pt p){return x*p.y-y*p.x;}
  bool operator<(pt p)const{ // for convex hull
  return x<p.x-EPS||(abs(x-p.x)<=EPS&&y<p.y-EPS);}
  bool left(pt p, pt q){ // is it to the left of directed line pq?
  return (q-p)%(*this-p)>EPS;}
  pt rot(pt r){return pt(*this%r,*this*r);}
  pt rot(double a){return rot(pt(sin(a),cos(a)));} };
pt ccw90(1,0); pt cw90(-1,0); int sgn2(double x){return x<0?-1:1;}
struct ln {
  pt p,pq; ln(pt p, pt q):p(p),pq(q-p){} ln(){}
  bool has(pt r){return dist(r)<=EPS;}
  bool seghas(pt r){return has(r)&&(r-p)*(r-(p+pq))<=EPS;}
  bool operator/(ln l){return abs(pq.unit()%l.pq.unit())<=EPS;}
  bool operator==(ln l){return *this/l&&has(l.p);}
  pt operator^(ln l){ // intersection
  if(*this/l)return pt(DINF,DINF); pt r=l.p+l.pq*((p-l.p)%pq/(l.pq%pq));
  return r; }
  double angle(ln l){return pq.angle(l.pq);}
  int side(pt r){return has(r)?0:sgn2(pq%(r-p));}
  pt proj(pt r){return p+pq*((r-p)*pq/pq.norm2());}
  pt ref(pt r){return proj(r)*2-r;}
  double dist(pt r){return (r-proj(r)).norm();}
  ln rot(auto a){return ln(p,p+pq.rot(a));} };
ln bisector(ln l, ln m){ // angle bisector
  pt p=l^m; return ln(p,p+l.pq.unit()+m.pq.unit()); }
ln bisector(pt p, pt q){ return ln((p+q)*.5,p).rot(ccw90); }
// polygon intersecting left side of halfplanes
struct halfplane:public ln{
  double angle; halfplane(){}
```

```
  halfplane(pt a,pt b){p=a; pq=b-a; angle=atan2(pq.y,pq.x);}
  bool operator<(halfplane b)const{return angle<b.angle;}
  bool out(pt q){return pq%(q-p)<-EPS;} };
vector<pt> intersect(vector<halfplane> b){
  vector<pt>bx={{DINF,DINF},{-DINF,DINF},{-DINF,-DINF},{DINF,-DINF}};
  fore(i,0,4) b.pb(halfplane(bx[i],bx[(i+1)%4]));
  sort(all(b)); int n=sz(b),q=1,h=0;
  vector<halfplane> c(sz(b)+10);
  fore(i,0,n){
  while(q<h&&b[i].out(c[h]^c[h-1])) h--;
  while(q<h&&b[i].out(c[q]^c[q+1])) q++; c[++h]=b[i];
  if(q<h&&abs(c[h].pq%c[h-1].pq)<EPS){
    if(c[h].pq*c[h-1].pq<=0) return {}; h--;
    if(b[i].out(c[h].p)) c[h]=b[i]; } }
  while(q<h-1&&c[q].out(c[h]^c[h-1]))h--;
  while(q<h-1&&c[h].out(c[q]^c[q+1]))q++;
  if(h-q<=1)return {}; c[h+1]=c[q]; vector<pt> s;
  fore(i,q,h+1) s.pb(c[i]^c[i+1]); return s; }
struct pol {
  int n;vector<pt> p; pol(){}
  pol(vector<pt> _p){p=_p;n=p.size();}
  double area(){
    double r=0.; fore(i,0,n)r+=p[i]%p[(i+1)%n];
    return abs(r)/2; // negative if CW, positive if CCW
  } };
```

5.5 latlong

```
struct ll{double r, lat, lon;};
struct rect{double x, y, z;};
ll convert(rect& P){ll Q;Q.r = sqrt(P.x*P.x+P.y*P.y+P.z*P.z);
  Q.lat = 180/M_PI*asin(P.z/Q.r);
  Q.lon = 180/M_PI*acos(P.x/sqrt(P.x*P.x+P.y*P.y)); return Q;}
rect convert(ll& Q){rect P;
  P.x = Q.r*cos(Q.lon*M_PI/180)*cos(Q.lat*M_PI/180);
  P.y = Q.r*sin(Q.lon*M_PI/180)*cos(Q.lat*M_PI/180);
  P.z = Q.r*sin(Q.lat*M_PI/180);return P;}
```

5.6 line

```
struct line {
  double a, b, c;
  line(point p, point q) {
  a = p.y - q.y; b = q.x - p.x; c = -a * p.x - b * p.y; };
  void setOrigin(point p) { c += a * p.x + b * p.y; } //trasladar linea
      como si p fuera el origen
};
double det(double a, double b, double c, double d) {
  return a * d - b * c; }
point intersec(line a, line b) { //primero estar seguro si no son
    paralelas
  double d = -det(a.a, a.b, b.a, b.b);
  return point(det(a.c, a.b, b.c, b.b) / d, det(a.a, a.c, b.a, b.c) / d);
      }
```

5.7 minkowski

```
typedef vector<point> poly;
void norm(poly &pol) {
  int pos = 0;
  fore(i, 0, pol.size()) { if(pol[i] < pol[pos]) pos = i; }
  rotate(pol.begin(), pol.begin() + pos, pol.end()); }
poly minkos(poly &a, poly &b) {
  norm(a); norm(b);
  int posa = 0, posb = 0, ta = a.size(), tb = b.size();
  poly res; ll cro;
  while(posa < ta || posb < tb) {
  res.pb(a[(posa) % ta] + b[(posb) % tb]);
  cro=(a[(posa+1)%ta]-a[posa % ta])^(b[(posb + 1) % tb]-b[posb%tb]);
  if(cro == 0) posa++, posb++;
  else if(cro < 0) posb++;
  else posa++; }
  return res; }
```

5.8 nearest neighbour

```
// Closest Neighbor - O(n * log(n))
const ll N = 1e6+3, INF = 1e18;
ll n, cn[N], x[N], y[N];
// number of points, closes neighbor, x coordinates, y coordinates
ll sqr(ll i) { return i*i; }
ll dist(int i, int j) { return sqr(x[i]-x[j]) + sqr(y[i]-y[j]); }
```

```
ll dist(int i) { return i == cn[i] ? INF : dist(i, cn[i]); }
bool cpx(int i, int j) { return x[i]<x[j]||(x[i]==x[j]&&y[i]<y[j]);}
bool cpy(int i, int j) { return y[i]<y[j]||(y[i]==y[j]&&x[i]<x[j]);}
ll calc(int i, ll x0) { ll dlt = dist(i) - sqr(x[i]-x0);
  return dlt >= 0 ? ceil(sqrt(dlt)) : -1;}
void updt(int i, int j, ll x0, ll &dlt) {
  if (dist(i) > dist(i, j)) cn[i] = j, dlt = calc(i, x0);}
void cmp(vi &u, vi &v, ll x0) {
  for(int a=0, b=0; a<u.size(); ++a) {
    ll i = u[a], dlt = calc(i, x0);
    while(b < v.size() and y[i] > y[v[b]]) b++;
    for(int j=b-1;j>=0&&y[i]-dlt<=y[v[j]];j--)updt(i,v[j],x0,dlt);
    for(int j=b;j<sz(v)&&y[i]+dlt>=y[v[j]];j++)updt(i,v[j],x0,dlt);}}
void slv(vi &ix, vi &iy) {int n = ix.size();
  if (n == 1) { cn[ix[0]] = ix[0]; return; }
  int m = ix[n/2]; vi ix1, ix2, iy1, iy2;
  for(int i=0; i<n; ++i) { if (cpx(ix[i], m)) ix1.push_back(ix[i]);
    else ix2.push_back(ix[i]);
    if (cpx(iy[i], m)) iy1.push_back(iy[i]); else iy2.push_back(iy[i]);}
  slv(ix1, iy1);slv(ix2, iy2);
  cmp(iy1, iy2, x[m]);cmp(iy2, iy1, x[m]);}
void slv(int n) {vi ix, iy;ix.resize(n);iy.resize(n);
  for(int i=0; i<n; ++i) ix[i] = iy[i] = i;
  sort(ix.begin(), ix.end(), cpx);
  sort(iy.begin(), iy.end(), cpy);slv(ix, iy);}
```

5.9 planar graph dual

```
vi g[tam];int n; // input graph (must be connected)
vi gd[tam];int nd; // output graph
vi nodes[tam]; // nodes delimiting region (in CW order)
map<ii,int> ps,es;
void get_dual(vector<pt> &p,vi &nods){ //p: points corresp to nodes
  ps.clear();es.clear();
  for(int x : nods){
    pt p1, p2;
    auto cmp = [&](int a, int b){
      p1 = p[a] - p[x], p2 = p[b] - p[x];
      a=p1.x>0&&p1.y==0||p1.y>0,b=p2.x>0&&p2.y==0||p2.y>0;
      if(a == b) return p1 % p2 > 0;
      return a < b; };
    sort(all(g[x]), cmp);
    fore(i,0,g[x].size())ps[{x,g[x][i]}]=i; }
  nd=0;
  for(int xx : nods)for(auto yy:g[xx])if(!es.count({xx,yy})){
    int x=xx,y=yy;gd[nd].clear();nodes[nd].clear();
    while(!es.count({x,y})){
      es[{x,y}]=nd;nodes[nd].pb(y);
      int z=g[y][(ps[{y,x}]+1)%g[y].size()];x=y;y=z; }
    nd++; }
  for(auto p:es){
    pair<int,int> q={p.f.s,p.f.f};
    assert(es.count(q));
    if(es[q]!=p.s)gd[p.s].pb(es[q]); }
  fore(i,0,nd){
    sort(all(gd[i]));
    gd[i].erase(unique(all(gd[i])),gd[i].end()); } }
```

5.10 point-in-poly

```
// logaritmico counterclockwise
bool inpol(poly &pol, point p) {
  int n = pol.size();
  if(((pol[1]-pol[0])^(p-pol[0]))<0||((pol[n - 1]-pol[0])^(p-pol[0]))>0)
    return 0;
  int lo = 1, hi = n - 2, mid, res;
  while(lo <= hi) {
    mid = (lo + hi) / 2;
    if(((pol[mid] - pol[0])^(p - pol[0])) >= 0) res = mid, lo=mid+1;
    else hi = mid - 1; }
  return ((pol[res + 1] - pol[res]) ^ (p - pol[res])) >= 0; }
```

6 Graph

6.1 2sat

```
namespace sat2{
  set<int> G[tam], Ginv[tam];
  int N, mark[tam], mark_comp[tam], valor[tam];
  int neg(const int& x) { return (x>=N)? x - N : x + N;}
  void add_(const int& x,const int& y) {G[x].insert(y);Ginv[y].insert(x);}
  void addor(const int x,const int y) {add_(neg(x),y);add_(neg(y),x);}
  void dfs0(int u, vector<int>& orden) { mark[u] = 1;
```

```
  for(auto& v: G[u]) {
    if (!mark[v]) dfs0(v,orden);
  }   orden.push_back(u); }
  void dfs1(int u, const int& cmp) { mark_comp[u] = cmp;
  for(auto& v: Ginv[u]) {
    if (!mark_comp[v]) dfs1(v,cmp); } }
  bool check() { bool impos = false;
  for(int i = 0; i < N; i++) {
    impos |= (mark_comp[i] == mark_comp[neg(i)]);
     valor[i] = (mark_comp[i] > mark_comp[neg(i)]) ;}
  return !impos; } }
```

6.2 bellman-ford

```
vector<int> adj[tam], adjw[tam];
int dist[N], v, w; dist[0] = 0;
for (int i = 0; i < n-1; ++i)
  for (int u = 0; u < n; ++u)
    for (int j = 0; j < adj[u].size(); ++j)
      v = adj[u][j], w = adjw[u][j],
      dist[v] = min(dist[v], dist[u]+w);
```

6.3 centroid

```
vi g[tam]; vector<vi> par(tam), dispa(tam);
vi siz(tam), cintra(tam);
void sisi(int node, int pa, int dis, int papazo) {
  if(papazo != -1) par[node].pb(papazo), dispa[node].pb(dis);
  siz[node] = 1;
  for(int x : g[node]) if(x != pa && !cintra[x])
    sisi(x, node, dis + 1, papazo), siz[node] += siz[x]; }
int centruida(int node, int pa, int simp) {
  for(int x : g[node])
    if(x != pa && !cintra[x])
      if(siz[x] > simp / 2)
        return centruida(x, node, simp);
  return node; }
void centauros(int node, int pa) {
  sisi(node, -1, 1, pa);
  int centris=centruida(node,-1,siz[node]); cintra[centris]=1;
  for(int x:g[centris]) if(!cintra[x]) centauros(x, centris); }
```

6.4 dominator$_{t}ree$

```
//idom[i]=parent of i in dominator tree with root=rt, or -1
int n,rnk[tam],pre[tam],anc[tam],idom[tam],semi[tam],low[tam];
vi g[tam],rev[tam],dom[tam],ord;
void dfspre(int pos){
  rnk[pos]=sz(ord); ord.pb(pos);
  for(auto x:g[pos]){
    rev[x].pb(pos); if(rnk[x]==n) pre[x]=pos,dfspre(x); } }
int eval(int v){
  if(anc[v]<n&&anc[anc[v]]<n){
    int x=eval(anc[v]);
    if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
    anc[v]=anc[anc[v]]; }
  return low[v]; }
void dominators(int rt){
  fore(i,0,n){
    dom[i].clear(); rev[i].clear();
    rnk[i]=pre[i]=anc[i]=idom[i]=n; semi[i]=low[i]=i; }
  ord.clear(); dfspre(rt);
  for(int i=sz(ord)-1;i;i--){
    int w=ord[i];
    for(int v:rev[w]){
      int u=eval(v); if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u]; }
    dom[semi[w]].pb(w); anc[w]=pre[w];
    for(int v:dom[pre[w]]){
      int u=eval(v); idom[v]=(rnk[pre[w]]>rnk[semi[u]]?u:pre[w]); }
    dom[pre[w]].clear(); }
  for(int w:ord) if(w!=rt&&idom[w]!=semi[w]) idom[w]=idom[idom[w]];
  fore(i,0,n) if(idom[i]==n)idom[i]=-1; else dom[idom[i]].pb(i); }
```

6.5 dynamic-connectivity

```
namespace dyn_con { struct UnionFind {
  int n,comp; vector<int> uf,si,c;
  UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1){
  fore(i,0,n)uf[i]=i;}
  int find(int x){return x==uf[x]?x:find(uf[x]);}
  bool join(int x, int y){
  if((x=find(x))==(y=find(y)))return false;
  if(si[x]<si[y])swap(x,y);
  si[x]+=si[y];uf[y]=x;comp--;c.pb(y); return true; }
```

```
  int snap(){return c.size();}
  void rollback(int snap){
  while(c.size()>snap){
    int x=c.back();c.pop_back(); si[uf[x]]-=si[x];uf[x]=x;comp++; }
  } };
enum {ADD,DEL,QUERY}; struct Query {int type,x,y;};
struct DynCon {
  vector<Query> q; UnionFind dsu; vector<int> mt;
  map<pair<int,int>,int> last; DynCon(int n):dsu(n){}
  void add(int x, int y){ if(x>y)swap(x,y);
    q.pb((Query){ADD,x,y});mt.pb(-1);last[{x,y}]=q.size()-1;
  }
  void remove(int x, int y){
    if(x>y)swap(x,y); q.pb((Query){DEL,x,y});
    int pr=last[{x,y}];mt[pr]=q.size()-1;mt.pb(pr); }
  void query(){q.pb((Query){QUERY,-1,-1});mt.pb(-1);}
  void process(){ // answers all queries in order
  if(!q.size())return;
  fore(i,0,q.size())if(q[i].type==ADD&&mt[i]<0)mt[i]=q.size();
  go(0,q.size()); }
  void go(int s, int e){
  if(s+1==e){
    if(q[s].type==QUERY) // answer query using DSU
    printf("%d\n",dsu.comp); // can ask current state UnionFind
    return; }
  int k=dsu.snap(),m=(s+e)/2;
  for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]<s)dsu.join(q[i].x,q[i].y);
  go(s,m);dsu.rollback(k);
  for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.join(q[i].x,q[i].y);
  go(m,e);dsu.rollback(k);
  }  }; }
```

6.6 edmonds-blossom

```
namespace ed_bls{ // undirected G
  vector<int> g[tam];
  int n,m,mt[tam],qh,qt,q[tam],ft[tam],bs[tam];
  bool inq[tam],inb[tam],inp[tam];
  int lca(int root, int x, int y){
  memset(inp,0,sizeof(inp));
  while(1){
    inp[x=bs[x]]=true; if(x==root)break; x=ft[mt[x]]; }
  while(1){ if(inp[y=bs[y]])return y; else y=ft[mt[y]]; } }
  void mark(int z, int x){
    while(bs[x]!=z){
      int y=mt[x]; inb[bs[x]]=inb[bs[y]]=true; x=ft[y];
      if(bs[x]!=z)ft[x]=y; } }
  void contr(int s, int x, int y){ int z=lca(s,x,y);
  memset(inb,0,sizeof(inb)); mark(z,x);mark(z,y);
  if(bs[x]!=z)ft[x]=y; if(bs[y]!=z)ft[y]=x;
  fore(x,0,n)if(inb[bs[x]]){ bs[x]=z;
    if(!inq[x])inq[q[++qt]=x]=true; } }
  int findp(int s){
  memset(inq,0,sizeof(inq)); memset(ft,-1,sizeof(ft));
  fore(i,0,n)bs[i]=i; inq[q[qh=qt=0]=s]=true;
  while(qh<=qt){
    int x=q[qh++]; for(int y:g[x])if(bs[x]!=bs[y]&&mt[x]!=y){
    if(y==s||mt[y]>=0&&ft[mt[y]]>=0)contr(s,x,y);
    else if(ft[y]<0){
      ft[y]=x; if(mt[y]<0)return y;
      else if(!inq[mt[y]])inq[q[++qt]=mt[y]]=true; } } }
  return -1; }
  int aug(int s, int t){
  int x=t,y,z;
  while(x>=0){ y=ft[x]; z=mt[y]; mt[y]=x;mt[x]=y; x=z; }
  return t>=0; }
  int edmonds(){ // O(n^2 m)
  int r=0; memset(mt,-1,sizeof(mt));
  fore(x,0,n)if(mt[x]<0)r+=aug(x,findp(x));
  return r; } }
```

6.7 erdos gallai

```
//Erdos-Gallai-O(nlogn) check if it's possible to create simple
//graph (undirected edges) from a sequence of vertice's degrees
bool gallai(vector<int> v) {
  vector<ll> sum; sum.resize(v.size());
  sort(v.begin(), v.end(), greater<int>()); sum[0] = v[0];
  for (int i = 1; i < v.size(); i++) sum[i] = sum[i-1] + v[i];
  if (sum.back() % 2) return 0;
  for (int k = 1; k < v.size(); k++) {
    int p = lower_bound(all(v), k, greater<int>()) - v.begin();
    if (p < k) p = k;
    if (sum[k-1] > 1ll*k*(p-1) + sum.back() - sum[p-1]) return 0; }
  return 1; }
```

6.8 eulerian-path

```
// Directed version (uncomment commented code for undirected)
struct edge { int y;
//  list<edge>::iterator rev;
  edge(int y):y(y){} };
list<edge> g[MAXN];
void add_edge(int a, int b){
  g[a].push_front(edge(b));/*auto ia=g[a].begin();
  g[b].push_front(edge(a));auto ib=g[b].begin();
  ia->rev=ib;ib->rev=ia;*/ }
vector<int> p;
void go(int x){
  while(g[x].size()) {int y=g[x].front().y;
  //g[y].erase(g[x].front().rev);
  g[x].pop_front(); go(y);} p.push_back(x);}
vector<int> get_path(int x){ // get a path that begins in x
// check that a path exists from x before calling to get_path!
  p.clear();go(x);reverse(p.begin(),p.end()); return p; }
```

6.9 HLD LCA con heavy paths

```
/*- poner grafo en G
- correr init(N)
- para cada nodo v->segBase[pos[v]]=val[v] (segBase es nuevo arreglo)
- inicializar segment tree para [0, N-1]
- implementar la funcion segQuery(int l, int r) */
vector<vi> G; vi parent, depth, heavy, head, pos; int curPos;
int dfs(int v) {
  int size = 1;int maxChildrenSize = 0;
  for (int c : G[v]) {
    if (c == parent[v]) continue; parent[c] = v;
    depth[c] = depth[v] + 1; int cSize = dfs(c); size += cSize;
    if (cSize > maxChildrenSize) {
      maxChildrenSize = cSize; heavy[v] = c; } }
  return size; }
void decompose(int v, int h) {
  head[v] = h; pos[v] = curPos++;
  if (heavy[v] != -1) decompose(heavy[v], h);
  for (int c : G[v])
    if (c != parent[v] && c != heavy[v]) decompose(c, c); }
void init(int nodes) {
    parent.assign(nodes, -1); depth.resize(nodes);
    heavy.assign(nodes, -1); head.resize(nodes);
    pos.resize(nodes); curPos = 0; dfs(0); decompose(0, 0); }
int segQuery(int l, int r);
int hldQuery(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]]) swap(a, b);
        int pathMax = segQuery(pos[head[b]], pos[b]);
        res = max(pathMax, res); }
    if (depth[a] > depth[b]) swap(a, b);
    int lastPath = segQuery(pos[a], pos[b]);
    res = max(res, lastPath); return res; }
```

6.10 max weight lca

```
vector <pii> adj[N];
int vis[N], h[N], anc[N][M], mx[N][M];
void dfs (int u) { vis[u] = 1;
  for (auto p : adj[u]) {
    int v = p.st, w = p.nd;
    if (!vis[v]) { h[v] = h[u]+1; anc[v][0] = u; mx[v][0] = w;
      dfs(v); } } }
void build () {
  // cl(mn, 63) -- Don't forget to initialize with INF if min edge!
  anc[1][0] = 1; dfs(1);
  for (int j = 1; j <= K; j++) for (int i = 1; i <= n; i++) {
    anc[i][j] = anc[anc[i][j-1]][j-1];
    mx[i][j] = max(mx[i][j-1], mx[anc[i][j-1]][j-1]); } }
int mxedge (int u, int v) { int ans = 0;
  if (h[u] < h[v]) swap(u, v);
  for (int j = K; j >= 0; j--) if (h[anc[u][j]] >= h[v]) {
    ans = max(ans, mx[u][j]); u = anc[u][j]; }
  if (u == v) return ans;
  for (int j = K; j >= 0; j--) if (anc[u][j] != anc[v][j]) {
    ans = max(ans, mx[u][j]); ans = max(ans, mx[v][j]);
    u = anc[u][j]; v = anc[v][j]; }
  return max({ans, mx[u][0], mx[v][0]}); }
```

6.11 tarjan y block cut tree

```
  int n;
```

```
vector<vector<int>> adj; vector<bool> visited;
vector<int> tin, low; int timer;
vector<vectotr<int>> comps; stack<int> stk;
void dfs(int v, int p = -1) {
  visited[v]=true;tin[v]=low[v]=timer++;stk.push(v);int children=0;
  for (int to : adj[v]) {
    if (to == p) continue;
    if (visited[to]) low[v] = min(low[v], tin[to]);
    else { dfs(to, v); low[v] = min(low[v], low[to]);
      if (low[to] >= tin[v]) {
        if (p != -1) IS_CUTPOINT(v); comps.push_back({v});
        while (comps.back().back() != to) {
          comps.back().push_back(stk.top()); stk.pop(); } }
      if (low[to] > tin[v]) IS_BRIDGE(v, to); ++children; } }
  if(p == -1 && children > 1) IS_CUTPOINT(v); }
void find_cutpoints() {
  timer = 0; visited.assign(n, false);
  tin.assign(n, -1); low.assign(n, -1);
  for (int i = 0; i < n; ++i) { if (!visited[i]) dfs(i); } }
vector<int> id; int curBCTNode; vector<vector<int> > tree;
vector<bool> isAP;
void buildTree() {
  curBCTNode = 0; id.assign(n, -1); tree.clear(); isAP.clear();
  fore(v, 0, n) {
    if (cutpoint[v]) {
      id[v] = tree.size(); tree.pb({}); isAP.pb(true); } }
  for (auto comp : comps) {
    int v = tree.size(); tree.pb({}); isAP.pb(false);
    for (int x : comp) {
      if (cutpoint[x]) { tree[v].pb(id[x]); tree[id[x]].pb(v); }
      else id[x] = v; } } }
```

7 Math

7.1 basis z2

```
int basis[30], si = 0;
void add(int x) {
  fore(i, 0, 21)  if(x & 1 << i) {
      if(basis[i]) x ^= basis[i];
      else { basis[i] = x; si++; return; } } }
```

7.2 berlekamp

```
/* Description: Recovers any n-order linear recurrence relation
from the first 2n terms of the recurrence. Useful for guessing
linear recurrences after brute-forcing the first terms. Should work
on any field, but numerical stability for floats is not guaranteed.
Output will have size  n.*/
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0; vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1; ll b = 1;
  fore(i,0,n) { ++m; ll d = s[i] % mod;
  fore(j,1,L+1) d = (d + C[j] * s[i - j]) % mod; if (!d) continue;
  T = C; ll coef = d * modpow(b, mod-2) % mod;
  fore(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
  if (2 * L > i) continue; L = i + 1 - L; B = T; b = d; m = 0; }
  C.resize(L+1); C.erase(C.begin()); for(ll& x : C) x=(mod-x)%mod;
  return C; }
```

7.3 catalan

```
//Catalan, parentesis balanceados, arboles binarios,
//triangulacion poligono convexto de n + 2 lados,
//caminos en grilla sin atravesar diagonal
//Cat[n] = C(2n, n) / (n + 1)
Cat[0] = 1; Cat[n+1] = Cat[n] * 2 * (2 * n + 1) / (n + 2);
Cat[n] = Cat[n-1] * 2 * (2 * n - 1) / (n + 1);
```

7.4 chinese-remainder

```
constexpr ll safe_mod(ll x, ll m) {
  x %= m; if (x < 0) x += m; return x; }
constexpr std::pair<ll, ll> inv_gcd(ll a, ll b) {
  a = safe_mod(a, b); if (a == 0) return {b, 0};
  ll s = b, t = a, m0 = 0, m1 = 1;
  while (t) {
  ll u = s / t; s -= t * u; m0 -= m1 * u; ll tmp = s;
  s = t; t = tmp; tmp = m0; m0 = m1; m1 = tmp; }
  if (m0 < 0) m0 += b / s; return {s, m0}; }
ii crt(vi& r, const vi& m) {
  assert(r.size()==m.size());int n=int(r.size()); ll r0=0,m0=1;
  for (int i = 0; i < n; i++) {
```

```
  assert(1 <= m[i]); ll r1=safe_mod(r[i], m[i]), m1 = m[i];
  if (m0 < m1) { swap(r0, r1); swap(m0, m1); }
  if (m0%m1==0) { if (r0%m1!=r1) return {0,0}; continue; }
  ll g, im; tie(g, im) = inv_gcd(m0, m1); ll u1 = (m1 / g);
  if ((r1 - r0) % g) return {0, 0};
  ll x = (r1 - r0) / g % u1 * im % u1; r0 += x * m0; m0 *= u1;
  if (r0 < 0) r0 += m0; } return {r0, m0}; }
```

7.5 count primes 23

```
struct _count_primes_struct_t_ {
  vi primes, mnprimes;
  ll ans, y, vector<pair<pair<ll, int>, char>> queries;
  ll count_primes(ll n) {
    y = pow(n, 0.64); if (n < 100) y = n;
    primes.clear(); mnprimes.assign(y + 1, -1); ans = 0;
    for (int i = 2; i <= y; ++i) {
      if (mnprimes[i] == -1) {
        mnprimes[i] = primes.size(); primes.push_back(i); }
      for (int k = 0; k < primes.size(); ++k) {
        int j = primes[k]; if (i * j > y) break; mnprimes[i * j] = k;
        if (i % j == 0) break; } }
    if (n < 100) return primes.size(); ll s = n / y;
    for (int p : primes) { if (p > s) break; ans++; }
    int ssz = ans, ptr = primes.size() - 1;
    for (int i = ssz; i < primes.size(); ++i) {
      while (ptr >= i && (ll)primes[i] * primes[ptr] > n) --ptr;
      if (ptr < i) break; ans -= ptr - i + 1;}
    phi(n, ssz - 1);
    sort(queries.begin(), queries.end());int ind = 2,sz = primes.size();
    fenwick fw(sz); for (auto [na, sign] : queries) {
      auto [n, a] = na; while (ind<=n)fw.add(sz-1-mnprimes[ind++], 1);
      ans += (fw.ask(sz - a - 2) + 1) * sign; }
    queries.clear(); return ans - 1; }
  void phi(ll n, int a, int sign = 1) { if (n == 0) return;
    if (a == -1) { ans += n * sign; return; }
    if (n<=y){queries.emplace_back(make_pair(n,a),sign);return;}
    phi(n, a - 1, sign); phi(n / primes[a], a - 1, -sign); }
  struct fenwick {vi tree;int n;
    fenwick(int n = 0) : n(n) {tree.assign(n, 0);}
    void add(int i, int k) {for(;i<n;i=(i|(i+1)))tree[i]+=k;}
    int ask(int r){intres=0;for(;r>=0;r=(r&(r+1))-1)
        res += tree[r];return res;}};} _count_primes_;
ll count_primes(ll n) {return _count_primes_.count_primes(n);}
```

7.6 exteded-euclid

```
void euclid(ll a, ll b, ll &x, ll &y) {
  if (b) euclid(b, a%b, y, x), y -= x*(a/b);
  else x = 1, y = 0; }
//find (x,y) t.q. a*x+b*y=c or return false if it's not possible
//[x + k*b/gcd(a, b), y - k*a/gcd(a, b)] are also solutions
bool diof(ll a, ll b, ll c, ll &x, ll &y){
  euclid(abs(a), abs(b), x, y); ll g = abs(__gcd(a, b));
  if(c % g) return false; x *= c / g; y *= c / g;
  if(a < 0) x = -x; if(b < 0) y = -y; return true; }
```

7.7 fast-gcd

```
int gcd(int a, int b) {
  if (!a || !b) return a | b;
  unsigned shift=__builtin_ctz(a|b); a>>=__builtin_ctz(a);
  do { b >>= __builtin_ctz(b); if (a > b) swap(a, b);
  b -= a; } while (b); return a << shift; }
```

7.8 formulas

```
//Stirling number of the second kind is the number of ways to partition a
    set of n objects into k non-empty subsets.
S(n,k)= ks (n-1,k)+S(n-1,k-1),where S(0,0)=1,s(n,0)=s(0,n)=0
S(n,2)=2^(n-1)-1
S(n,k) k !=number of ways to color n nodes using colors from 1 to k such
    that each color is used at least once.
An r-associated Stirling number of the second kind is the number of ways
    to partition a set of n objects into k subsets,
with each subset containing at least r elements.It is denoted by Sr(n,k)
    and obeys the recurrence relation.
Sr(n+1,k)=kSr(n,k)+C(n,r-1)Sr(n-r+1,k-1)
//The Stirling numbers of the first kind count permutations according to
    their number of cycles (counting fixed points as cycles of length
    one).
```

```
S(n,k) counts the number of permutations of n elements with k disjoint
    cycles.
S(n,k) = (n-1)S(n-1,k)+S(n-1,k-1),where,s(0,0) =1,S(n,0)=s(0,n)=0
Sum(k,0,n) S(n,k) = n!
The unsigned Stirling numbers may also be defined algebraically, as the
    coefficient of the rising factorial:
    x^(~n)=x(x+1)(x+n1)=Sum(k,0,n)s(n,k)x^k
//Bell number count the number of partition of a set
Bn+1 = Sum(k,0,n){C(n,k)*Bk}
Bn = S Sum(k,0,n)Sr(n,k), where Sr is Stirling number of 2kind
//Formally, for a sequence of numbers {ai}, we define the ordinary
    generating function (OGF) of a to be A(x)=Sum(i,0,inf)aix^i.
1/(1 x ) = 1+ x + x^2+...= Sum(n,0,inf)x^n
 ln (1 x )=x + x^2/2 + x^3/3+...=Sum(n,0,inf)x^n/n
e^x=1+x + x^2/2! + x^3/3!+...=Sum(n,0,inf)x^n/n!
(1 x )^ k =C( k1 ,0)x^0+C(k,1)x^1+C(k+1,2)x^2+...=Sum(n,0,inf)C(n+k-1,n)x^n
For OGF, C(x)=A(x)^k generates the sequence
    cn=Sum(i1...ik,i1+i2+...+ik=n)(ai1*ai2...*aik)
For EGF, C(x)=A(x)^k generates the sequence
cn=Sum(i1...ik,i1+i2+...+ik=n)(ai1*ai2...*aik)*n!/(i1!*...ik!)
Suppose want to generate the sequence cn=a0+a1+...+an. Then, we can take
    C(x)=1/(1x)*A(x).
```

7.9 gauss mod prime

```
//ll A[N][M+1], X[M]
for(int j=0; j<m; j++) { int l = j;
for(int i=j+1; i<n; i++)if(A[i][j]%p)l=i;
for(int k = 0; k < m+1; k++) swap(A[l][k],A[j][k]);
for(int i = j+1; i < n; i++) {ll t=mulmod(A[i][j],inv(A[j][j],p),p);
for(int k = j; k < m+1; k++) A[i][k]=(A[i][k]-mulmod(t,A[j][k],p)+p)%p;}}
for(int i = m-1; i >= 0; i--) { for(int j = m-1; j > i; j--)
    A[i][m] = (A[i][m] - mulmod(A[i][j],X[j],p)+p)%p;
    X[i] = mulmod(A[i][m],inv(A[i][i],p),p);}
```

7.10 gauss

```
// returns (0,1,or  ). if there's a solution, it's in ans
const double EPS = 1e-9; const int INF = 2;
int gauss (vector<vector<double>> a, vector<double> &ans) {
  int n = (int) a.size(); int m = (int) a[0].size() - 1;
  vector<int> where(m, -1);
  for (int col=0, row=0; col<m && row<n; ++col) {
    int sel = row;
    for (int i=row; i<n; ++i)
      if (abs (a[i][col]) > abs (a[sel][col])) sel = i;
    if (abs (a[sel][col]) < EPS) continue;
    for(int i=col;i<=m;++i) swap(a[sel][i],a[row][i]);
    where[col] = row;
    for (int i=0; i<n; ++i)
      if (i != row) { double c = a[i][col] / a[row][col];
        for(int j=col;j<=m;++j) a[i][j]-=a[row][j]*c; } ++row; }
  ans.assign (m, 0);
  for (int i=0; i<m; ++i)
    if (where[i] != -1) ans[i] = a[where[i]][m] / a[where[i]][i];
  for (int i=0; i<n; ++i) { double sum = 0;
    for (int j=0; j<m; ++j) sum += ans[j] * a[i][j];
    if (abs (sum - a[i][m]) > EPS) return 0; }
  for (int i=0; i<m; ++i) if (where[i] == -1) return INF;
  return 1; }
```

7.11 interpol-o(n)

```
// evaluar un "polinomio interpolado" en o(nlogMOD)
// debe cumplir xi+1 -   xi   = xj +1   - xj   for all i,   j   < n
// recibe vector de ys tal que f(i) = y[i]
ll eval(vll ys, ll x) {
  int n = ys.size(); if(x < n) return ys[x];
  ll res = 0, up = 1, dow = 1;
  fore(i,1,n) dow=dow*(MOD-i)%MOD,up=up*(x-i)%MOD;
  fore(i, 1, n) {
  up = up * (x - i + 1) % MOD * pot(x - i, MOD - 2) % MOD;
  dow = dow * i % MOD * pot(MOD - (n - i), MOD - 2) % MOD;
  res = (res+ys[i]*up%MOD*pot(dow, MOD - 2) % MOD) % MOD; }
  return res; }
```

7.12 josephus

```
/* Josephus Problem - It returns the position to be, in order to not die.
    O(n)*/
/* With k=2, for instance, the game begins with 2 being killed and then
    n+2, n+4, ... */
```

```
ll josephus(ll n, ll k) {
  if(n==1) return 1;
  else return (josephus(n-1, k)+k-1)%n+1;
}
/* Another Way to compute the last position to be killed - O(d * log n) */
ll josephus(ll n, ll d) {
  ll K = 1;
  while (K <= (d - 1)*n) K = (d * K + d - 2) / (d - 1);
  return d * n + 1 - K;
}
```

7.13 linear sieve

```
vi prime; bool iscom[MAXN];
void sieve (int n) { fill(iscom, iscom + n, false);
  for (int i = 2; i < n; ++i) {
    if (!iscom[i]) prime.push_back (i);
    for (int j = 0; j < prime.size () && i*prime[j] < n; ++j) {
      iscom[i*prime[j]] = true; if (i%prime[j]==0) break; } } }
```

7.14 LinearRecurrence

```
/*Generates the kth term of an n-order linear recurrence
S[i] =    S[i    j    1]tr[j], given S[0... n1 ] and tr[0... n1 ].Faster
than matrix multiplication. Useful together with BerlekampMassey.
Usage: linearRec({0,1},{1,1},k) kth Fibonacci number O(n^2logk)*/
typedef vector<ll> Poly;
  ll linearRec(Poly S, Poly tr, ll k) { int n = sz(tr);
  auto combine = [&](Poly a, Poly b) { Poly res(n * 2 + 1);
  fore(i,0,n+1) fore(j,0,n+1)
    res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
  for (int i = 2 * n; i > n; --i) fore(j,0,n)
    res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
  res.resize(n + 1); return res;};
  Poly pol(n + 1), e(pol); pol[0] = e[1] = 1;
  for (++k; k; k /= 2) { if (k % 2) pol = combine(pol, e);
    e = combine(e, e); }
  ll res = 0; fore(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res; }
```

7.15 matrix-determinant

```
const double EPS=1e-4;
double reduce(vector<vector<double> >& x){
  int n=x.size(),m=x[0].size();
  int i=0,j=0;double r=1.;
  while(i<n&&j<m){ int l=i;
  fore(k,i+1,n)if(abs(x[k][j])>abs(x[l][j]))l=k;
  if(abs(x[l][j])<EPS){j++;r=0.;continue;}
  if(l!=i){r=-r;swap(x[i],x[l]);} r*=x[i][j];
  for(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
  fore(k,0,n){ if(k==i)continue;
    for(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]*x[i][l]; }
  i++;j++; } return r; }
```

7.16 matrix-fast-pow

```
typedef vector<vector<ll> > Matrix;
Matrix ones(int n) {
  Matrix r(n,vector<ll>(n)); fore(i,0,n)r[i][i]=1;
  return r; }
Matrix operator*(Matrix &a, Matrix &b) {
  int n=SZ(a),m=SZ(b[0]),z=SZ(a[0]);
  Matrix r(n,vector<ll>(m));
  fore(i,0,n)fore(j,0,m)fore(k,0,z)
  r[i][j]+=a[i][k]*b[k][j],r[i][j]%=mod; return r; }
Matrix be(Matrix b, ll e) {
  Matrix r=ones(SZ(b));
  while(e){if(e&1LL)r=r*b;b=b*b;e/=2;} return r; }
```

7.17 moebius

```
//f(n)=sum(d|n,g(d))=>g(n)=sum(d|n,f(d)*mu(n/d))
//f(d)=sum(i->inf,g(i*d)*mu(i));f(d)=#func(a)->d;g(d)=#d|func(a)
//sum(d|n,mu(d))=e(n)=[n=1]
int mu[tam], is_prime [tam];
fore(i, 0, tam) mu[i]=is_prime[i]=1;
fore(i, 2, tam) if(is_prime[i]) {
  forg(j, i, tam, i) {
    if(j > i) is_prime[j] = 0; if(j / i % i == 0) mu[j]=0;
    mu[j] = -mu[j]; } }
```

7.18 pollard-rho-miller-rabil

```
ll gcd(ll a, ll b){return a?gcd(b%a,a):b;}
ll mulmod(ll a, ll b, ll m) {
  ll r=a*b-(ll)((long double)a*b/m+.5)*m; return r<0?r+m:r; }
ll expmod(ll b, ll e, ll m){
  if(!e)return 1; ll q=expmod(b,e/2,m);q=mulmod(q,q,m);
  return e&1?mulmod(b,q,m):q; }
bool is_prime_prob(ll n, int a){
  if(n==a)return true; ll s=0,d=n-1;
  while(d%2==0)s++,d/=2; ll x=expmod(a,d,n);
  if((x==1)||(x+1==n))return true;
  fore(_,0,s-1){ x=mulmod(x,x,n); if(x==1)return false;
  if(x+1==n)return true; } return false; }
bool rabin(ll n){ // true iff n is prime
  if(n==1)return false; int ar[]={2,3,5,7,11,13,17,19,23};
  fore(i,0,9)if(!is_prime_prob(n,ar[i]))return false;
  return true; }
const int MAXP=1e6+1; // sieve size
int sv[MAXP]; // sieve
ll add(ll a, ll b, ll m){return (a+=b)<m?a:a-m;}
ll rho(ll n){ static ll s[MAXP];
  while(1){
  ll x=rand()%n,y=x,c=rand()%n; ll *px=s,*py=s,v=0,p=1;
  while(1){ *py++=y=add(mulmod(y,y,n),c,n);
    *py++=y=add(mulmod(y,y,n),c,n); if((x=*px++)==y)break;
    ll t=p; p=mulmod(p,abs(y-x),n);
    if(!p)return gcd(t,n);
    if(++v==26){ if((p=gcd(p,n))>1&&p<n)return p; v=0; } }
  if(v&&(p=gcd(p,n))>1&&p<n)return p; } }
void init_sv(){
  fore(i,2,MAXP)if(!sv[i])for(ll j=i;j<MAXP;j+=i)sv[j]=i; }
void fact(ll n, map<ll,int>& f){ // call init_sv first!!!
  for(auto&& p:f){ while(n%p.f==0){ p.s++; n/=p.f; } }
  if(n<MAXP)while(n>1)f[sv[n]]++,n/=sv[n];
  else if(rabin(n))f[n]++;
  else {ll q=rho(n);fact(q,f);fact(n/q,f);} }
```

7.19 stirling y bell

```
// stir[n][k] # formas de partir set tam n en k conjuntos
// bell[n] # de formas de partir un conjunto
ll stir[tam][tam]; bell[tam];
void stirBell() {
  fore(i, 1, tam) { stir[i][1] = 1; fore(j, 2, 1010)
  stir[i][j] = (j * stir[i - 1][j] %MOD+stir[i-1][j-1])%MOD; }
  fore(i,1,tam)fore(j,1,i+1)bell[i]=(bell[i]+stir[i][j])%MOD; }
```

7.20 tonelly shanks

```
ll legendre(ll a, ll p){
  if(a%p==0)return 0; if(p==2)return 1;
  return fpow(a,(p-1)/2,p); }
ll tonelli_shanks(ll n, ll p){ // sqrt(n) mod p (p must be a prime)
  assert(legendre(n,p)==1); if(p==2)return 1;
  ll s=__builtin_ctzll(p-1), q=(p-1LL)>>s, z=rnd(1,p-1);
  if(s==1)return fpow(n,(p+1)/4LL,p);
  while(legendre(z,p)!=p-1)z=rnd(1,p-1);
  ll c=fpow(z,q,p), r=fpow(n,(q+1)/2,p), t=fpow(n,q,p), m=s;
  while(t!=1){ ll i=1, ts=(t*t)%p; while(ts!=1)i++,ts=(ts*ts)%p;
    ll b=c; fore(_,0,m-i-1)b=(b*b)%p; r=r*b%p;c=b*b%p;t=t*c%p;m=i;
  } return r; }
```

8 other

8.1 dates

```
int dateToInt(int y, int m, int d){
  return 1461*(y+4800+(m-14)/12)/4+367*(m-2-(m-14)/12*12)/12-
    3*((y+4900+(m-14)/12)/100)/4+d-32075;
}
void intToDate(int jd, int& y, int& m, int& d){
  int x,n,i,j;x=jd+68569;
  n=4*x/146097;x-=(146097*n+3)/4;
  i=(4000*(x+1))/1461001;x-=1461*i/4-31;
  j=80*x/2447;d=x-2447*j/80;
  x=j/11;m=j+2-12*x;y=100*(n-49)+i+x;
}
int DayOfWeek(int d, int m, int y){ //starting on Sunday
  static int ttt[]={0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
```

```
  y-=m<3;return (y+y/4-y/100+y/400+ttt[m-1]+d)%7; }
```

8.2 discrete log

```
//Returns min x for which a^x%m=b%m, a and m are coprime.
int solve(int a, int b, int m) {
  a %= m, b %= m; int n = sqrt(m) + 1; int an = 1;
  for (int i = 0; i < n; ++i) an = (an * 1ll * a) % m;
  unordered_map<int, int> vals;
  for (int q = 0, cur = b; q <= n; ++q) {
    vals[cur] = q; cur = (cur * 1ll * a) % m; }
  for (int p = 1, cur = 1; p <= n; ++p) {
    cur = (cur * 1ll * an) % m;
    if (vals.count(cur)) {
      int ans=n*p-vals[cur]; return ans; } } return -1; }
```

8.3 harmonic lemma

```
for (int i = 1, la; i <= n; i = la + 1)
  la = n / (n / i); //n/x yields the same value for i<=x<=la.
```

8.4 mo

```
void remove(idx);void add(idx);int get_answer();int block_size;
struct Query { int l, r, idx;
  bool operator<(Query other) const {
    if (l / block_size != other.l / block_size)
      return mp(l, r) < mp(other.l, other.r);
    return ((l / block_size) & 1)?(r<other.r):(r>other.r); } };
vector<int> mo_s_algorithm(vector<Query> queries) {
  vector<int> answers(queries.size());
  sort(queries.begin(), queries.end());
  int cur_l = 0; int cur_r = -1;
  for (Query q : queries) {
    while (cur_l > q.l) { cur_l--; add(cur_l); }
    while (cur_r < q.r) { cur_r++; add(cur_r); }
    while (cur_l < q.l) { remove(cur_l); cur_l++; }
    while (cur_r > q.r) { remove(cur_r); cur_r--; }
    answers[q.idx] = get_answer(); } return answers; }
```

8.5 parallel binary search

```
struct Query { int i, ans; /*+ query related info*/ };
vector<Query> req;
void pbs(vector<Query>& qs,int l/*=minv*/,int r/*=maxv*/) {
  if (qs.empty()) return;
  if (l==r) { for(auto&q:qs)req[q.i].ans=l;return; }
  int mid = (l + r) / 2;
  //mid = (l + r + 1) / 2 if different simple upper/lower bound
  for (int i = l; i <= mid; i++) { /*add val to ds*/ }
  vector<Query> vl, vr; for (auto& q : qs) {
    if (/* cond */) vl.push_back(q); else vr.push_back(q); }
  pbs(vr, mid + 1, r);
  for (int i = l; i <= mid; i++) { /*remove val from ds*/ }
  pbs(vl, l, mid); }
```

8.6 pragmas cubanos

```
#pragma GCC target("avx,avx2,sse3,ssse3,sse4.1,sse4.2,tune=native")
#pragma GCC optimize(3)
#pragma GCC optimize("O3")
#pragma GCC optimize("inline")
#pragma GCC optimize("-fgcse")
#pragma GCC optimize("-fgcse-lm")
#pragma GCC optimize("-fipa-sra")
#pragma GCC optimize("-ftree-pre")
#pragma GCC optimize("-ftree-vrp")
#pragma GCC optimize("-fpeephole2")
#pragma GCC optimize("-fsched-spec")
#pragma GCC optimize("-falign-jumps")
#pragma GCC optimize("-falign-loops")
#pragma GCC optimize("-falign-labels")
#pragma GCC optimize("-fdevirtualize")
#pragma GCC optimize("-fcaller-saves")
#pragma GCC optimize("-fcrossjumping")
#pragma GCC optimize("-fthread-jumps")
#pragma GCC optimize("-freorder-blocks")
#pragma GCC optimize("-fschedule-insns")
#pragma GCC optimize("inline-functions")
#pragma GCC optimize("-ftree-tail-merge")
#pragma GCC optimize("-fschedule-insns2")
#pragma GCC optimize("-fstrict-aliasing")
#pragma GCC optimize("-falign-functions")
```

```
#pragma GCC optimize("-fcse-follow-jumps")
#pragma GCC optimize("-fsched-interblock")
#pragma GCC optimize("-fpartial-inlining")
#pragma GCC optimize("no-stack-protector")
#pragma GCC optimize("-freorder-functions")
#pragma GCC optimize("-findirect-inlining")
#pragma GCC optimize("-fhoist-adjacent-loads")
#pragma GCC optimize("-frerun-cse-after-loop")
#pragma GCC optimize("inline-small-functions")
#pragma GCC optimize("-finline-small-functions")
#pragma GCC optimize("-ftree-switch-conversion")
#pragma GCC optimize("-foptimize-sibling-calls")
#pragma GCC optimize("-fexpensive-optimizations")
#pragma GCC optimize("inline-functions-called-once")
#pragma GCC optimize("-fdelete-null-pointer-checks")
```

8.7 prime numbers

```
   2    3    5    7   11   13   17   19   23   29
  31   37   41   43   47   53   59   61   67   71
  73   79   83   89   97  101  103  107  109  113
 127  131  137  139  149  151  157  163  167  173
 179  181  191  193  197  199  211  223  227  229
 233  239  241  251  257  263  269  271  277  281
 283  293  307  311  313  317  331  337  347  349
 353  359  367  373  379  383  389  397  401  409
 419  421  431  433  439  443  449  457  461  463
 467  479  487  491  499  503  509  521  523  541
 547  557  563  569  571  577  587  593  599  601
 607  613  617  619  631  641  643  647  653  659
 661  673  677  683  691  701  709  719  727  733
 739  743  751  757  761  769  773  787  797  809
 811  821  823  827  829  839  853  857  859  863
 877  881  883  887  907  911  919  929  937  941
 947  953  967  971  977  983  991  997 1009 1013
1019 1021 1031 1033 1039 1049 1051 1061 1063 1069
1087 1091 1093 1097 1103 1109 1117 1123 1129 1151
1153 1163 1171 1181 1187 1193 1201 1213 1217 1223
1229 1231 1237 1249 1259 1277 1279 1283 1289 1291
1297 1301 1303 1307 1319 1321 1327 1361 1367 1373
1381 1399 1409 1423 1427 1429 1433 1439 1447 1451
1453 1459 1471 1481 1483 1487 1489 1493 1499 1511
1523 1531 1543 1549 1553 1559 1567 1571 1579 1583
1597 1601 1607 1609 1613 1619 1621 1627 1637 1657
1663 1667 1669 1693 1697 1699 1709 1721 1723 1733
1741 1747 1753 1759 1777 1783 1787 1789 1801 1811
1823 1831 1847 1861 1867 1871 1873 1877 1879 1889
1901 1907 1913 1931 1933 1949 1951 1973 1979 1987

     970'997     971'483  921'281'269  999'279'733
1'000'000'009 1'000'000'021 1'000'000'409 1'005'012'527
10005512850571600911
```

8.8 python fast in

```
import sys
inputs = sys.stdin.read().splitlines()
con = 1
ln = 0
while True:
        a, b = map(int, inputs[ln].split())
        ln += 1
        # a = int(input())
        # b = int(input())
        if a == 0:
                break
        su = 0
        for i in range (a):
                su += int(inputs[ln])
                ln += 1
        print(f"Bill #{con} costs {su}: each friend should pay {su // b}")
        print()
        con += 1
```

8.9 python

```
# reopen
import sys
sys.stdout = open('out','w')
sys.stdin = open('in' ,'r')

//Dummy example
R = lambda: map(int, input().split())
n, k = R(),
```

```
v, t = [], [0]*n
for p, c, i in sorted(zip(R(), R(), range(n))):
    t[i] = sum(v)+c
    v += [c]
    v = sorted(v)[::-1]
    if len(v) > k:
        v.pop()
print(' '.join(map(str, t)))
```

8.10 shortcuts

```
// Better random mt19937_64 para 64 bits
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
cout << rng() << endl;
shuffle(permutation.begin(), permutation.end(), rng);
ios::sync_with_stdio(0); cin.tie(0); cout.tie(0);
freopen("asd.txt", "r", stdin);freopen("qwe.txt", "w", stdout);
// while TLE
double t = clock(), TLE = 3;
while((clock() - t) / CLOCKS_PER_SEC < TLE);
// ordered_set
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef
    tree<int,null_type,less<int>,rb_tree_tag,tree_order_statistics_node_update>
    ordered_set;
// find_by_order kth largest 0 indexed, order_of_key finds how many are
    less than
// Faster map gp_hash_table<int,int,my_hash> m;
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
struct my_hash {
  const uint64_t RANDOM =
      chrono::steady_clock::now().time_since_epoch().count();
  static uint64_t splitmix64(uint64_t x) {
  x += 0x9e3779b97f4a7c15;
  x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
  x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
  return x ^ (x >> 31); }
  size_t operator()(uint64_t x) const {
  return splitmix64(x + RANDOM); } };
```

8.11 simulate analing

```
vi pref; for(int i=1;i<=n;i++)pref.pb(i);
shuffle(pref.begin(),pref.end(),rng);
int res=costo(pref);vi respuesta=resi;double tempereatura=1e6;
while(clock() / (double) CLOCKS_PER_SEC <= 0.975749){
  int a=uniform_int_distribution<int>(0,n-1)(rng);
  int b=uniform_int_distribution<int>(0,n-2)(rng);
  if(a==b){ b++; if(b==n)b=0;}vi next=pref;
  swap(next[a],next[b]);/*funcion vecino*/int nextCosto=costo(next);
  if(nextCosto<res){ res=nextCosto;respuesta=resi;pref=next;}
  else{double proba=exp((res-nextCosto)/tempereatura);
    double r=uniform_real_distribution<double>(0,1)(rng);
    if(r<proba){res=nextCosto;respuesta=resi;pref=next;}}
  tempereatura*=0.9999;}
```

8.12 theo

```
 ErdsGallai   theorem
A sequence of non-negative integers d1>=...>=d_n can be represented
as the degree sequence of a finite simple graph on n vertices if and only
    if
d1+...+dn is even and sum(i=1->k, di) <= k*(k - 1) + sum(i=k+1->n,
    min(di, k))
holds for every k in 1<=k<=n
teorema de Erds-Szekeres
Para r, s dados, cualquier sucesin de longitud al menos (r-1)(s-1)+1
    contiene
una subsucesin montonamente creciente de longitud r o una subsucesin
montonamente decreciente de longitud s
The harmonic lemma
(n/1, n/2, ..., n/n) has at most 2*sqrt(n) different elements
greatest x that satisfy d(x) = d(i) is n / (n / i)
(Tutte) A graph, G = (V, E), has a perfect matching if and only if for
    every subset U of V, the subgraph induced by V - U has at most |U|
    connected components with an odd number of vertices.
Petersens Theorem. Every cubic, bridgeless graph contains a perfect
    matching.
(Dilworth) In any finite partially ordered set, the maximum number of
    elements in any antichain equals the minimum number of chains in any
    partition of the set into chains
Pick: A=I+B/2-1 (area of polygon, points inside, points on border)
```

```
(de Brujin) (k!)^(k(n-1))/k^n, # of de Brujin sequences, remove
    fractional part to consider rotated strings
```

9 Strings

9.1 aho-corasick

```
struct vertex {
  int go[26], pch, par, link = -1, super = -1, leaf = 0;
  vertex(): link(0), super(0) { mem(go, -1); }
  vertex(int ch, int from): pch(ch), par(from) { mem(go, -1); }
};
vector<vertex> t(1);
void add(string &s, int pos) { int node = 0;
  for(char ch : s) { ch -= 'a'; if(t[node].go[ch] == -1)
      t[node].go[ch] = t.size(); t.emplace_back(ch, node);
    node = t[node].go[ch]; } t[node].leaf = 1; }
int go(int node, char c);
int suff(int node) { if(t[node].link == -1)
    t[node].link=t[node].par==0?0:go(suff(t[node].par),t[node].pch);
  return t[node].link; }
int go(int node, char ch) { if(t[node].go[ch] == -1)
    t[node].go[ch] = node == 0 ? 0 : go(suff(node), ch);
  return t[node].go[ch]; }
int super(int v) { if(t[v].super == -1)
    t[v].super = t[suff(v)].leaf ? suff(v) : super(suff(v));
  return t[v].super; }
```

9.2 algoritmo-z

```
vector<int> alz(string s) {
  int n = s.size(); vector<int> z(n, 0);
  for(int i = 1, l = 0, r = 0; i < n; i++) {
    if(i <= r) z[i] = min(z[i - l], r - i + 1);
    while(i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if(r < i + z[i] - 1) l = i, r = i + z[i] - 1; } return z; }
```

9.3 Dinamic Aho Corasick

```
const int alpha = 26; int node[20][tam][alpha];
struct ahoCorasick {
  int root, nnode, link[tam], cnt[tam], id; bool dead;
  void init(int idx) {
    dead=false;id=idx;root=0;nnode=0;mem(node[id][root],0);
    fore(i, 0, nnode + 1) cnt[i] = 0; }
  void clear() { dead = true; }
  void insert(string &s, int val) { int len=s.size(),now=root;
    fore(i, 0, len) {
      int nxt = s[i] - 'a';
      if(!node[id][now][nxt]) { node[id][now][nxt] = ++nnode;
        mem(node[id][nnode],0); cnt[nnode] = 0; }
      now=node[id][now][nxt]; } cnt[now] += val; }
  void insertdict(vector<string> &dict, vector<int> &vals) {
    fore(i, 0, sz(dict)) insert(dict[i], vals[i]); }
  void pushLinks() { queue<int> Q; link[root]=-1;Q.push(root);
    while(!Q.empty()) { int u = Q.front(); Q.pop();
      fore(i, 0, alpha) { if(!node[id][u][i]) continue;
        int v = node[id][u][i]; int l = link[u];
        while(l != -1 && !node[id][l][i]) l = link[l];
        if(l!=-1) link[v] = node[id][l][i]; else link[v] = 0;
        cnt[v] += cnt[link[v]]; Q.push(v); } } }
  int query(string &p) { int u = root, ret = 0;
    for(char ch : p) { int nxt = ch - 'a';
      while(u != -1 && !node[id][u][nxt]) u = link[u];
      if(u == -1) u = 0; else u = node[id][u][nxt];
      ret += cnt[u]; } return ret; } } aho[20];
vector<string> dict[20]; vi vals[20];
void add(string &s, int val) { dict[0].pb(s); vals[0].pb(val);
  fore(i, 0, 20) { if(sz(dict[i]) > (1<<i)) {
      for(auto it : dict[i]) dict[i + 1].pb(it);
      for(auto it : vals[i]) vals[i + 1].pb(it);
      dict[i].clear(); vals[i].clear(); aho[i].clear(); }
    else {aho[i].init(i); aho[i].insertdict(dict[i], vals[i]);
      aho[i].pushLinks(); break; } } }
```

9.4 hsh-128

```
#define bint __int128
struct Hash {
  bint MOD=212345678987654321LL,P=1777771,PI=106955741089659571LL;
  vector<bint> h,pi; Hash(string& s){
```

```
  assert((P*PI)%MOD==1);h.resize(s.size()+1);
  pi.resize(s.size()+1);h[0]=0;pi[0]=1; bint p=1;
  fore(i,1,s.size()+1){
    h[i]=(h[i-1]+p*s[i-1])%MOD; pi[i]=(pi[i-1]*PI)%MOD;
    p=(p*P)%MOD; } } ll get(int s, int e){
  return (((h[e]-h[s]+MOD)%MOD)*pi[s])%MOD; } };
```

9.5 manacher

```
int d1[MAXN];//d1[i] = max odd pal centered on i
int d2[MAXN];//d2[i] = max even pal centered on i
//s  aabbaacaabbaa d1 1111117111111 d2 0103010010301
void manacher(string& s){ int l=0,r=-1,n=s.size();
  fore(i,0,n){ int k=i>r?1:min(d1[l+r-i],r-i);
    while(i+k<n&&i-k>=0&&s[i+k]==s[i-k])k++; d1[i]=k--;
    if(i+k>r)l=i-k,r=i+k; }
  l=0;r=-1; fore(i,0,n){
    int k=i>r?0:min(d2[l+r-i+1],r-i+1);k++;
    while(i+k<=n&&i-k>=0&&s[i+k-1]==s[i-k])k++; d2[i]=--k;
    if(i+k-1>r)l=i-k,r=i+k-1; } }
```

9.6 preffix-function

```
vector<int> kmp(string s) {
       int n = s.size(); vi pi(n, 0);
       for(int i = 1; i < n; i++) { int j = pi[i-1];
              while(j > 0 && s[j] != s[i]) j = pi[j-1];
              if(s[j] == s[i]) j++; pi[i] = j; } return pi; }
int aut[tam][26];
void genAut(string &s) { mem(aut, 0); vi pi = kmp(s);
  int n = s.size();
  fore(i, 0, n) aut[i][s[i]-'a'] = i+1; fore(i, 0, n+1) {
    for (int c = 0; c < 26; c++) {
      if (aut[i][c]) continue;
      if (i < n && s[i] == 'a' + c) aut[i][c] = i+1;
      else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
      else aut[i][c] = 0; } } }
```

9.7 suffix-array

```
vector<vi> table;
vi suffixa(string &s){
  int n = sz(s),cc,ax;vi sa(n),sa1(n),col(n),col1(n),head(n);
  fore(i, 0, n) sa[i] = i;
  auto cmp = [&](int a, int b){ return s[a] < s[b]; };
  stable_sort(sa.begin(),sa.end(),cmp);head[0]=col[sa[0]]=cc=0;
  fore(i, 1, n){ if(s[sa[i]] != s[sa[i-1]]) cc++, head[cc] = i;
    col[sa[i]] = cc;} table.pb(col);
  for(int k = 1; k < n; k *= 2){ fore(i, 0, n){
      ax = (sa[i] - k + n) % n; sa1[head[col[ax]]++] = ax; }
    swap(sa, sa1); col1[sa[0]] = head[0] = cc = 0;
    fore(i, 1, n){
    if(col[sa[i]]!=col[sa[i-1]]||col[(sa[i]+k)%n]!=col[(sa[i-1]+k)%n])
      cc++, head[cc] = i;
    col1[sa[i]] = cc;}
    swap(col,col1);table.pb(col);if(col[sa[n-1]]==n-1)break;}
  return sa; }
pair<int, int> query(int b, int e){
  int lev=31-__builtin_clz(e-b+1);
  return mp(table[lev][b],table[lev][e-(1<<lev)+1]); }
bool comp(int b1, int e1, int b2, int e2){
  int siz = min(e1 - b1, e2 - b2);
  ii le = query(b1, b1 + siz), ri = query(b2, b2 + siz);
  if(le == ri) return e1 - b1 < e2 - b2; return le < ri; }
vector<int> lcp(string &s, vector<int> &sa){
  int n = s.size(), k, z = 0; vector<int> sa1(n), lcp(n);
  fore(i, 0, n) sa1[sa[i]] = i;
  fore(i, 0, n){
    k = sa1[i];
    if(k < n - 1) while(s[i + z] == s[sa[k+1] + z]) z++;
    lcp[k] = z; z = max(z-1, 0); } return lcp; }
```